

# Graph Anomaly Detection in Time Series: A Survey

Thi Kieu Khanh Ho , Ali Karami , and Narges Armanfard 

(Survey Paper)

**Abstract**—With the recent advances in technology, a wide range of systems continue to collect a large amount of data over time and thus generate time series. Time-Series Anomaly Detection (TSAD) is an important task in various time-series applications such as e-commerce, cybersecurity, vehicle maintenance, and healthcare monitoring. However, this task is very challenging as it requires considering both the *intra-variable dependency* (relationships within a variable over time) and the *inter-variable dependency* (relationships between multiple variables) existing in time-series data. Recent graph-based approaches have made impressive progress in tackling the challenges of this field. In this survey, we conduct a comprehensive and up-to-date review of TSAD using graphs, referred to as G-TSAD. First, we explore the significant potential of graph representation for time-series data and its contributions to facilitating anomaly detection. Then, we review state-of-the-art graph anomaly detection techniques, mostly leveraging deep learning architectures, in the context of time series. For each method, we discuss its strengths, limitations, and the specific applications where it excels. Finally, we address both the technical and application challenges currently facing the field, and suggest potential future directions for advancing research and improving practical outcomes.

**Index Terms**—Time-series anomaly detection, graphs, deep learning, time-series signals, dynamic social networks, videos.

## I. INTRODUCTION

A TIME series is defined as an ordered sequence of values that represent the evolution of one variable, aka *univariate*, or multiple variables, aka *multivariate*, over time [1]. Time-series data is also referred to as time-stamped data, i.e., each data point or each observation corresponds to a specific time index. As time is a constituent of everything that is observable, time-series data can be found everywhere on the Earth, such as in nature (e.g., the wind speed, the temperature), in marketing and industrial activities (e.g., the stock price), in medicine (e.g., the heart and brain activity), and in surveillance and security (e.g., video streams, network traffic logs). Therefore, time-series data and its analysis have attracted significant interest in the past decade.

Received 28 April 2024; revised 26 March 2025; accepted 29 April 2025. Date of publication 2 May 2025; date of current version 3 July 2025. This work was supported in part by the Natural Sciences, Engineering Research Council of Canada (NSERC), in part by Fonds de recherche du Québec, in part by the Department of Electrical and Computer Engineering, McGill University, in part by Calcul Québec, and in part by the Compute Canada. Recommended for acceptance by B. Rosenhahn. (Corresponding author: Thi Kieu Khanh Ho.)

The authors are with McGill University, Montreal, QC H3A 0G4, Canada, and also with Mila-Quebec AI Institute, Montreal, QC H2S 3H1, Canada (e-mail: thi.k.ho@mail.mcgill.ca; ali.karami@mail.mcgill.ca; narges.armanfard@mcgill.ca).

Digital Object Identifier 10.1109/TPAMI.2025.3566620

Time-Series Anomaly Detection (TSAD), the process of detecting unusual patterns that do not conform to the expected behavior, has been widely studied [2]. The unusual patterns can be found in many real-world applications such as ecosystem disturbances in earth sciences, structural defects in jet turbine engineering, suspicious activities in surveillance videos, heart failure in cardiology, or seizures in brain activity. Most existing TSAD algorithms are designed to analyze time-series data as a univariate modality [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17] or as multiple individual modalities (variables) [18], [19], [20], [21], [21], [22], [23], [24]. In other words, these algorithms only consider the intra-variable information.

In many practical systems, a more proper and reliable system modeling can be obtained if the data are considered as multiple related modalities (variables) that are influenced by each other. Specifically, a comprehensive anomaly detection (AD) algorithm carefully shall model the variables' interaction over time, i.e., the inter-variable dependencies, as well as the intra-variable dependencies. Explicit consideration of the inter-variable dependencies allows capturing a system's spatial dependencies across variables. Examples of systems, which exhibit spatial dependencies, are the brain where the recording sensors are placed across the brain, and each sensor is considered as a variable; in a single time-series signal where every sample (feature) or time-interval (window) is considered as a variable; and in a video-based system where a variable can be assigned to every frame.

While many TSAD algorithms have addressed intra-variable dependencies, capturing inter-variable dependencies remains under-explored in existing studies. In recent years, graph-based methods have gained popularity in TSAD due to their ability to represent complex data structures inherent in time-series data. Unlike traditional tabular representations, graphs have demonstrated superior performance in AD by modeling inter-variable dependencies, where each variable is represented as a graph node and the interactions between variables are depicted as graph edges [25], [26]. This property is valuable for TSAD. For instance, if one variable experiences a change, related variables are likely to exhibit similar changes as well. This enables the identification of anomalies that may not be evident through conventional TSAD methods.

This survey explores TSAD using graph-based approaches. It categorizes existing research based on the types of constructed graphs, the nature of graph-based anomalies in time-series data, the detection methods employed, and the application domains.

TABLE I  
COMPARISON OF THE EXISTING SURVEYS WITH OURS

| Survey      | Year | GA | GAD | TSAD | G-TSAD |
|-------------|------|----|-----|------|--------|
| [27]        | 2020 | *  | —   | —    | —      |
| [26]        | 2021 | *  | —   | —    | —      |
| [47]        | 2021 | *  | —   | —    | —      |
| [25]        | 2022 | *  | —   | —    | —      |
| [28]        | 2022 | *  | —   | —    | —      |
| [29]        | 2022 | *  | —   | —    | —      |
| [30]        | 2022 | *  | —   | —    | —      |
| [31]        | 2022 | *  | —   | —    | —      |
| [32]        | 2022 | *  | —   | —    | —      |
| [33]        | 2023 | *  | —   | —    | —      |
| [34]        | 2024 | *  | —   | —    | —      |
| [35]        | 2024 | *  | —   | —    | —      |
| [36]        | 2021 | —  | *   | —    | —      |
| [37]        | 2022 | —  | *   | —    | —      |
| [38]        | 2023 | —  | *   | —    | —      |
| [39]        | 2023 | —  | *   | —    | —      |
| [40]        | 2023 | —  | *   | —    | —      |
| [41]        | 2024 | —  | *   | —    | —      |
| [42]        | 2024 | —  | *   | —    | —      |
| [43]        | 2019 | —  | —   | *    | —      |
| [44]        | 2021 | —  | —   | *    | —      |
| [2]         | 2021 | —  | —   | *    | —      |
| [45]        | 2023 | —  | —   | *    | —      |
| [46]        | 2024 | —  | —   | *    | —      |
| <b>Ours</b> | 2024 | —  | —   | —    | *      |

\* Indicates that the topic is reviewed in the corresponding survey study, — indicates otherwise.

*Paper Organization.* Section II presents an overview of existing surveys in the literature and our contributions. Section III outlines the challenges in time-series data. Sections IV and V provide foundational insights into graphs, common types of anomalies within graphs, and how the graph-based methods can address the time-series challenges. Section VI introduces our unified taxonomy for TSAD using graphs. Within each category, we discuss the strength of each method in addressing the challenges outlined in Section III, as well as its applications and technical drawbacks and how the subsequent methods mitigate these shortcomings. Section VII summarizes widely used datasets and their real-world applications. Section VIII discusses employed evaluation metrics in the field. Section IX discusses the current research challenges and promising directions for future research. Finally, we conclude our paper in Section X.

## II. OVERVIEW OF RELATED SURVEYS

Although graphs have proven effective and yielded state-of-the-art results in TSAD, as summarized in Table I, *none of existing surveys have focused on the emergence and applications of graphs for detecting anomalies in time-series data*, referred to as G-TSAD. For instance, Graph-based Approaches (GA) surveys [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35] cover a broad range of graph structure learning techniques for various tasks such as classification, clustering, and prediction, without focusing specifically on AD. On the other hand, Graph Anomaly Detection (GAD) surveys [36], [37], [38], [39], [40], [41], [42] provides a comprehensive review of graph-based methods for detecting various types of abnormal graph objects. However, both GA and GAD surveys solely review the graph-based methods applicable to static data (e.g., static

social networks or images) and do not provide comprehensive reviews on the methods that are able to capture the intra-variable dependencies existing in time-series data. Meanwhile, TSAD surveys [2], [43], [44], [45], [46] focus on methodologies for detecting anomalies within time-series signals, disregarding other data types of the time series such as dynamic social networks and videos. Additionally, existing TSAD surveys show the ability of non-graph deep learning methods to detect anomalies only based on the intra-variable dependencies, ignoring the inter-variable dependencies; this is while both intra- and inter-variable dependencies are crucial for TSAD.

The contributions of this survey are as follows:

- *The first survey in G-TSAD.* To the best of our knowledge, this is the first survey that reviews the state-of-the-art graph-based techniques for TSAD. Our primary goal of this survey is to showcase how anomaly detectors, which leverage graphs, detect anomalies in time-series data. Until now, all the relevant surveys focus on either GA, TSAD, or GAD. There is no dedicated and comprehensive survey on G-TSAD. Our work bridges this gap, and we expect that such structured and comprehensive survey helps to push forward the research in this active area.
- *A new perspective.* We introduce new concepts in the field of G-TSAD, such as the intra- and inter-variable dependencies, and constructed graphs for time-series data. Such concepts would pave the road for future research to provide well-structured and easy-to-understand G-TSAD algorithms.
- *A detailed overview of G-TSAD.* We discuss the key challenges in TSAD, the core motivations of employing graph representation for time-series data and its role in facilitating anomaly detection, and the various types of anomalies in graphs.
- *A systematic and comprehensive review.* We conduct a comprehensive and up-to-date review of the state-of-the-art on G-TSAD methods, primarily leveraging deep learning architectures, and categorize them into four groups to improve their clarity and accessibility. These categories include Autoencoder (AE)-based methods, Generative Adversarial Network (GAN)-based methods, predictive-based methods, and self-supervised methods. For each category, we describe technical details, highlight their advantages and disadvantages, and compare them.
- *Outlook on future directions.* We point out both technical and application limitations of the current studies and suggest promising directions for future works on G-TSAD.

## III. TIME-SERIES CHALLENGES

In general, a  $K$ -variable time series dataset can be denoted by  $X = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(K)})$ , where each  $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_N^{(i)})$ ,  $X \in \mathbb{R}^{K \times N \times m}$ ,  $\mathbf{x}^{(i)} \in \mathbb{R}^{N \times m}$ ,  $K$  is the number of variables,  $N$  is the number of observations, and  $m$  is the feature length, representing the number of features in each observation. Each  $x_j^{(i)}$ ,  $j \in \{1, \dots, N\}$  and  $i \in \{1, \dots, K\}$ , is an  $m$  dimensional vector corresponding to a recording at the  $j$ th time interval and the  $i$ th variable. For example, in Fig. 1 (Left),

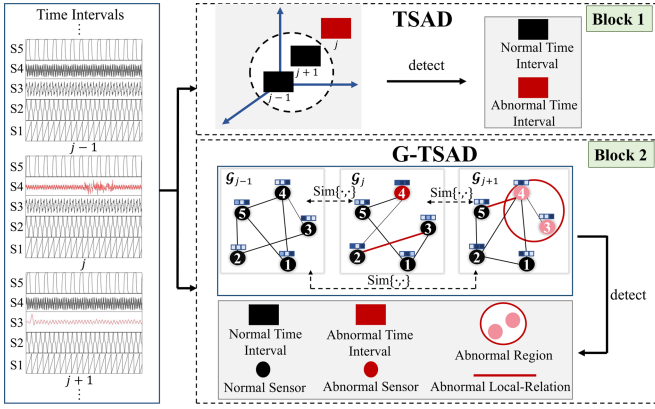


Fig. 1. An example of anomaly detection in a multivariate time-series signal data to show the difference between TSAD (Block 1) and G-TSAD (Block 2). The inputs are three successive time intervals (S: Sensor). In the constructed graphs, the solid and dash lines, respectively, indicate the inter-variable and intra-variable dependencies,  $m = 3$ , and the edge features are not shown for simplicity. Normal and abnormal cases are respectively shown in black and red colors. G-TSAD has the potential to detect anomalous sensors, local-relations, regions, and time intervals.

the raw input data of three intervals of a dataset  $X$  consisting of five variables (sensors) is shown. The raw interval at the  $i$ th variable is encoded by  $m = 3$  features, as depicted on the top of each sensor in Block 2 of Fig. 1. Hence,  $X$  can be represented as  $X = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(5)})$ , where  $X \in \mathbb{R}^{5 \times 3 \times m}$ ,  $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_{j-1}^{(i)}, x_j^{(i)}, x_{j+1}^{(i)}, \dots, x_N^{(i)})$ ,  $\mathbf{x}^{(i)} \in \mathbb{R}^{3 \times m}$ . Note that one may directly use the raw values as features; e.g., if an observation consists of 100 samples, then  $m = 100$ . Similarly, a video sequence can be considered as either univariate or multivariate data. For univariate data, one may consider every frame or a subset of frames as one observation of that single variable, and use the raw pixel values of the frame(s) or their corresponding features. In the multivariate case, each pixel can be considered as a variable. In this context, an observation of a variable can be obtained from a single frame or a subset of frames.

Working with time-series data requires careful consideration of several factors, which are discussed below in detail.

**Intra-variable dependency.** Observations available in one variable depend on each other, i.e., there exist dependencies of the  $j$ th observation and a lagged version of itself in the  $i$ th variable. Positive dependencies indicate that an increase (or decrease) in the  $j$ th observation is likely caused by an increase (or decrease) in the previous observations and to be followed by an increase (or decrease) in the subsequent observations of the  $i$ th variable. In contrast, negative dependencies suggest an inverse relationship. However, in practice, one of the main obstacles is dealing with the non-linear complex dependencies present within a variable. In fact, the mutual relationship between observations is not straightforward, and the effect of the future/past observations on the current one may vary over time. This makes it difficult to determine the appropriate lags for consideration, as different lags may have different levels of significance.

**Inter-variable dependency.** It is important to capture the inter-variable dependency as it can provide insights into the

underlying relationship patterns that exist between different variables; a useful property for TSAD. For example, if two variables are correlated, a change in one variable can be used to predict changes in another variable. Another example is the case if individual variables have fairly normal behavior, but when all (or a subset) of variables and their interactions are considered together, an anomaly can be caught. Hence, employing graphs that can capture complex relationships between variables would be a great tool for TSAD.

**Non-stationarity.** It is important to consider trends, seasonality, and unpredictability. Although the trend is known as an average tendency of the series, the trend itself may vary over different time intervals. Seasonality presents the variations of repeating short-term cycles in the series, which causes a change of variance over time. Unpredictability occurs randomly that are neither systematic nor predictable. Such properties influence the variables statistical properties such as mean, variance, etc. Thus, time-series data is said to be non-stationary, which may easily mislead an AD method as anomalies at certain observations may not be true anomalies [9]. Methods that can adapt to the changes in the data structures require a large number of training data.

**Dimensionality.** The current advances in technology allow us to record a large amount of time-series data to capture the intra-variable and inter-variable dependencies. Existing of such rich datasets allows us to design time-series analyses that are consistent and reliable across various datasets. Looking at the raw data, every data sample in every variable can be considered as one dimension; hence we are dealing with the curse of dimensionality challenge. Therefore, there is a need to develop algorithms that can handle such complicated and high-dimensional data.

**Noise.** It is important to make a distinction between noise and anomalies. Noise is the random, unwanted variation that degrades the data quality. In contrast, an anomaly is an unusual entity with respect to a set of pre-established normal observations. Therefore, AD algorithms explicitly designed to distinguish noise from anomalies are more robust when dealing with contaminated data; however, various types of noises may happen depending on the operational environment, recording sensor types, etc., that a researcher may not be aware at the problem onset. Hence, it is critical to understand the nature of noise in every application and apply appropriate noise modeling/reduction techniques to not mix them with the anomaly concept.

In the subsequent sections, we will delve into graph-based techniques and their effectiveness in tackling individual challenges or combinations thereof present in time-series data, hence enhancing our understanding of their applicability and utility in real-world scenarios.

#### IV. WHY GRAPHS?

##### A. Graph-Based Solutions for TSAD Challenges

1) **Capturing Intra- and Inter-Variable Dependencies:** Many TSAD studies have shown superior performance when employing graphs. This is because many real-world systems can be represented as graphs, and graphs can inherently capture



*intra- and inter-variable dependencies.* We define a graph set representing a time-series data as:

$$\mathbb{G} = \left\{ \mathcal{G}_j, \text{Sim}\{\mathcal{G}_j, \mathcal{G}_{j'}\} \right\}_{\substack{j, j' \in \{1, \dots, N\} \\ j \neq j'}}, \quad (1)$$

where  $\mathcal{G}_j = \{\mathbf{M}_j, \mathbf{A}_j\}$  is a graph representing the  $j$ th observation,  $\mathbf{M}_j$  is the  $K \times m$  node-feature matrix,  $\mathbf{A}_j$  is the  $K \times K \times m'$  edge-feature matrix representing the relationships across nodes (variables) in the  $j$ th observation,  $m$  and  $m'$  respectively denotes the number of features representing the nodes and their relations.  $\text{Sim}\{\cdot, \cdot\}$  defines the relation between two graphs. The  $\text{Sim}\{\cdot, \cdot\}$  goal is to capture the intra-variable dependencies between two observations. This can be defined as a function of node-features and edge-features, i.e.,  $\text{Sim}\{\mathcal{G}_j, \mathcal{G}_{j'}\} = \mathcal{F}\{\mathbf{M}_j, \mathbf{M}_{j'}, \mathbf{A}_j, \mathbf{A}_{j'}\}$ . The literature identifies two types of constructed graphs: static and dynamic, which are described as below.

*Dynamic Graphs.*  $\mathbb{G}$  is trained *dynamically* if either of  $\mathcal{G}_j$  or  $\text{Sim}\{\cdot, \cdot\}$  has learnable parameters; e.g., an encoder network with a set of parameters  $\xi_1$  can be used to provide  $\mathbf{M}_j$  and  $\mathbf{A}_j$  from the raw data that forms the  $j$ th observation [48], [49], or a network with a set of trainable parameters  $\xi_2$  can be used as  $\mathcal{F}$  [50], [51].

*Static Graphs.* If there are no learnable parameters assigned to  $\mathcal{G}_j$  and/or  $\text{Sim}\{\mathcal{G}_j, \mathcal{G}_{j'}\}$ , we are dealing with *static* graphs [52]. In other words, a static graph is built using a set of pre-defined node features and edges obtained from the  $j$ th observation of the  $K$  variables. For example, with prior knowledge about the sensors' locations in a  $K$ -variable sensory system, a static graph at the  $j$ th observation is constructed where nodes are sensors and edges are defined by the euclidean distance between sensors. Note that in some algorithms, the relations across static graphs are learned implicitly through employing a temporal-based network such as recurrent neural networks [53]. Notably, [53] has been shown to be the first in the G-TSAD field to incorporate a predefined  $\text{Sim}\{\cdot, \cdot\}$ , as an additional form of supervision within their model. This study demonstrates that utilizing  $\text{Sim}\{\cdot, \cdot\}$  improves model performance compared to approaches that do not make use of this measure.

2) *Handling Non-Stationarity in Time-Series Data:* In many real-world applications, dynamically learning graphs is crucial since dynamic graphs allow for the adaptation of relationships between variables as the underlying data distribution changes. For example, in financial markets, the correlations between assets (variables) may fluctuate over time due to shifting market conditions. By dynamically updating the graph edges and  $\text{Sim}\{\cdot, \cdot\}$  to reflect these changing relationships, dynamic graphs can effectively capture the *non-stationarity* inherent in time-series data.

3) *Addressing High Dimensionality:* Graphs have also demonstrated their capability to address other challenges such as *dimensionality* [34]. One approach is to construct the static graphs based on prior knowledge [52], incorporating information such as the distance, correlations and similarity measures between variables. Based on their relevance and importance in the graph structure, graph-based methods can select features that are most discriminative or relevant for the task at

hand. By focusing on a reduced set of features, dimensionality can be effectively managed while maintaining the predictive power of the data. Another strategy is to project the graph elements onto the graph embedding space using techniques such as node2vec [54] or graph neural networks [27], which can map high-dimensional data into a lower-dimensional space while preserving the structure and relationships encoded in the graph. Lastly, graph-based regularization techniques [55], [56], [57], [58] can be applied to impose constraints on the model parameters during the graph learning process. This ensures that the learned graphs exhibit desirable properties or structures of the graphs such as smoothness, where features should change smoothly between neighboring nodes, or sparsity, to prevent overly connected graphs.

4) *Mitigating Noise in Time-Series Data:* Graph regularization techniques can also mitigate the effect of *noise* in the data [59], [60], [61]. For example, smoothness regularization encourages gradual changes in signal values between neighboring nodes, effectively filtering out high-frequency noise components [62]. Moreover, many methods have been developed to minimize the impact of noisy data when learning the graph structure [63], [64], [65]. These algorithms leverage robust optimization techniques to handle noise, resulting in more accurate graph representations that are less susceptible to noise.

## B. Graph Representations Across Different Time-Series Domains

Fig. 1 visualizes G-TSAD versus the non-graph approach (i.e., TSAD). Fig. 1 (Left) shows raw signals from three time intervals of a multivariate system with five sensors, each recording different types of data with varying ranges and characteristics. Each sensor is then represented as a 3-dimensional feature vector, shown in both Blocks 1 and 2. Block 1 illustrates non-graph TSAD methods, where each interval is depicted as a rectangle containing data from all sensors. A key limitation of these conventional methods is their inability to explicitly model inter-sensor relationships. That is, they simply concatenate the sensor data, train the model based on this concatenated input, and aim to detect only interval-level anomalies. This diminishes their ability to detect anomalies at a finer-grained level, such as abnormal sensors, abnormal sensor interactions (aka relations), or anomalies involving multiple sensors (aka regions).

Moreover, Fig. 1 (Left) shows that the recording of Sensor #4 during the  $j$ th interval is abnormal. While, as shown in Block 1, TSAD methods fail to identify the location of the anomaly, G-TSAD methods can localize such node anomalies by analyzing each sensor individually, shown in Block 2. In contrast, the recordings of Sensors #3 and #4 during the  $j + 1$ th interval do not exhibit significant anomalies detectable by non-graph TSAD methods. However, a well-designed G-TSAD method can identify these anomalies by considering node relationships within and between intervals through edges and  $\text{Sim}\{\cdot, \cdot\}$ , as discussed in Section IV.

As depicted in Fig. 1, graphs hold potential to detect various types of anomalies in a sensory multivariate system. One may wonder whether graphs can be applied to an univariate

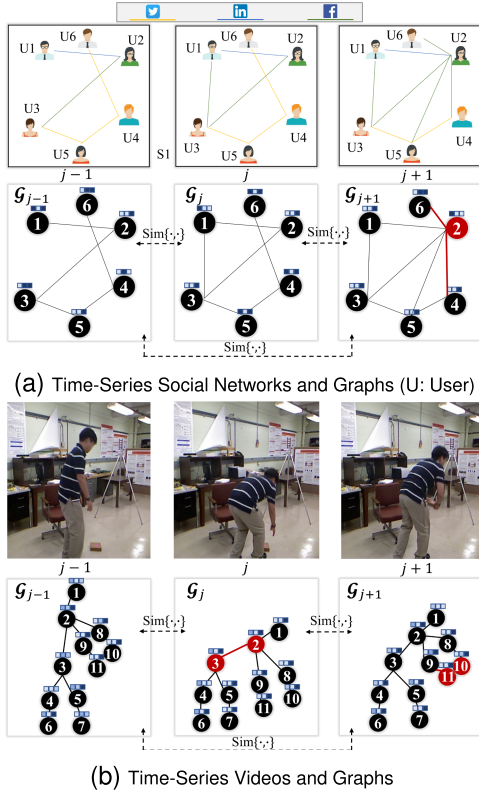


Fig. 2. Examples of time-series data and the corresponding constructed graphs. Each example is shown with three consecutive observations. The top figures show the original data and the bottom figures show the constructed graphs. In the constructed graphs, the solid and dash lines, respectively, indicate the inter-variable and intra-variable dependencies,  $m = 3$ , and the edge features are not shown for simplicity. Normal and abnormal cases are respectively shown in black and red colors.

time-series signal system, where there is only a single sensor used to record the signals, in order to detect various anomaly types such as point-level, contextual-level and trend-level anomalies defined in [44]. Constructing graphs that represent the relationships between data points, short/long time intervals, with each denoted as a variable, can be beneficial. By analyzing the level of connectivity within the graph, anomalies can be identified as data points or time intervals that have few or weak connections to neighboring points, contexts, or patterns.

Graphs also present a capability in other time-series domains. Fig. 2 shows two additional examples of time-series data and their constructed graphs by which both the intra- and inter-variable information are captured. Note that the three successive observations at the  $j - 1$ ,  $j$ , and  $j + 1$  intervals, shown in Fig. 2(a) and (b) respectively, correspond to three successive snapshots of a social network and three successive frames of a video. In social networks, the users are considered as graph nodes, and the users' interactions as edges. Most normal nodes have reasonable number of connections, which indicates the user has an ordinary social network activity. A node with an exceptionally high number of connections can be defined as an abnormal user. An abnormal node may represent a celebrity, an influential person, or a leader within the network (Fig. 2(a)). In video applications, a video can be modeled as

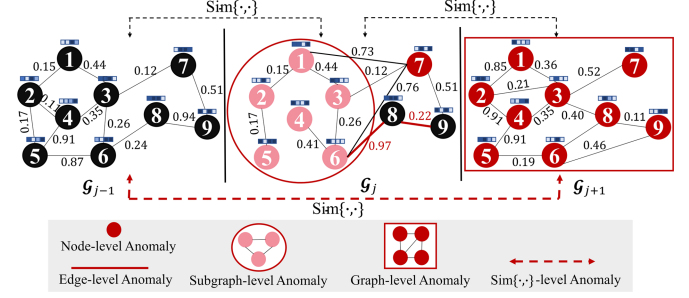


Fig. 3. Example of node-level, edge-level, sub-graph-level, graph-level, and  $\text{Sim}\{\cdot, \cdot\}$ -level anomalies in a graph set  $\mathbb{G}$  with three successive observations. In each graph  $\mathcal{G}$ ,  $m = 4$  and  $m' = 1$ .

a stream of time-evolving object-level graphs, where an object (e.g., a human body joint, an object in the scene) is considered as a graph node and edges represent the nodes relation within a video frame. Fig. 2(b) shows an example of anomalous activity detection where a node is assigned to every body joint – any unexpected joint movements shall be detected as anomalies.

In summary, we have shown the capability of graphs in addressing various challenges present in time-series data described in Section III. More technical insights into methods specifically tailored to address individual challenges or combinations thereof are provided in Section VI.

## V. ANOMALIES IN GRAPHS

As illustrated in Section IV, there are many types of anomalies in time-series data. For instance, anomalies may manifest as anomalous sensors, local relationships between sensors, regions, and time intervals within a sensory multivariate signal system. In a univariate time-series system, anomalies can take the form of point-level, contextual-level, and trend-level anomalies. Dynamic social networks may feature abnormal users (nodes) and relationships between users (edges) as anomalies. In video streams, unusual object movements can be regarded as anomalies. To provide a general categorization applicable to most of time-series data, we categorize anomalies that can be observed in graphs into five groups, namely anomalous nodes, edges, sub-graphs, graphs and  $\text{Sim}\{\cdot, \cdot\}$ . They are, respectively, known as variables, the local relations between variables, small sets of variables and their corresponding connections, the full set of variables and their corresponding connections at the  $j$ th observation, and the global relations between observations. Note that detecting anomalous objects in constructed graphs is much more challenging than other graph types due to the dynamic variations of nodes and edges [25].

Fig. 3 shows a graph set with three successive observations, i.e.,  $\mathbb{G} = \left\{ \mathcal{G}_j, \text{Sim}\{\mathcal{G}_j, \mathcal{G}_{j'}\} \right\}_{j \neq j'}$ . The five anomaly types are illustrated in the figure. In the first observation, all nodes and edges are normal. However, in the second observation, one anomalous node, two anomalous edges, and one anomalous sub-graph are appeared. In an anomalous sub-graph, each node and edge might look normal; however, if they are considered as a group, an anomaly can be detected. As sub-graphs vary in size,

TABLE II  
SUMMARY OF REPRESENTATIVE AE-BASED METHODS

| Method           | Year | Data Type       | Constructed Graph | Learning Task | Technique                                          | Anomaly Type | Evaluation Metric      | Targeted Application                  |
|------------------|------|-----------------|-------------------|---------------|----------------------------------------------------|--------------|------------------------|---------------------------------------|
| DeepSphere [68]  | 2018 | Signals, Videos | Dynamic           | Node, Edge    | LSTM-AE with hypersphere learning                  | Node         | Kappa Statistics, RMSE | Transportation, Social Media          |
| OmniAnomaly [74] | 2019 | Signals         | Dynamic           | Node, Edge    | GRU-VAE with planar NF layers                      | Node         | Prec, Rec, F1          | Server Machines, Satellite Monitoring |
| DVGRN [72]       | 2022 | Signals         | Dynamic           | Node, Edge    | GNN-VAE with a Gaussian-distributed channel module | Node         | Prec, Rec, F1          | Server Machines, Satellite Monitoring |
| GReLeN [70]      | 2022 | Signals         | Dynamic           | Node, Edge    | GNN-VAE with a self-attention module               | Node         | Prec, Rec, F1          | Water Systems, Server Machines        |
| GANF [48]        | 2022 | Signals         | Dynamic           | Node, Edge    | LSTM-NF with Directed Acyclic Graphs               | Node         | AUC                    | Highway Traffic, Power Grids          |
| CAN [75]         | 2023 | Signals         | Dynamic           | Node, Edge    | GNN with a couple attention module                 | Graph        | Prec, Rec, F1          | Water, Soil Moisture Systems          |
| TSAD-C [76]      | 2024 | Signals         | Dynamic           | Node, Edge    | Diffusion models with a graph modeling module      | Graph        | F1, Rec, APR           | Server Machines, Biomedical Systems   |

as well as node and edge level properties, detecting anomalous sub-graphs is more challenging than nodes and edges. Regarding graph-level anomalies, they are defined as abnormal graphs in a graph stream – specifically, given a sequence of graphs, an anomalous graph at the third observation can be distinguished from other graphs based on their unusual evolving pattern of nodes, edges and corresponding features.

Lastly, anomalous  $\text{Sim}\{\cdot, \cdot\}$  occurs when there are unusual relationships between graphs (aka observations). As  $\text{Sim}\{\cdot, \cdot\}$  can capture both short-term and long-term relations across observations, anomalous  $\text{Sim}\{\cdot, \cdot\}$  can be detected by analyzing the evolution of the graphs and their relationships over time. Assume that the three observations in Fig. 3 correspond to the monthly stock market; short-term variations, modeled by  $\text{Sim}\{\mathcal{G}_{j-1}, \mathcal{G}_j\}$  and  $\text{Sim}\{\mathcal{G}_j, \mathcal{G}_{j+1}\}$ , may represent a small price change. These small variations can be considered normal. However, a longer relationship presented in  $\text{Sim}\{\mathcal{G}_{j-1}, \mathcal{G}_{j+1}\}$  may raise an abnormality.

In general, given a graph  $\mathbb{G}$ , the goal of an ideal anomaly detector system is to produce a learnable anomaly scoring function  $f(\cdot)$  that assigns an anomaly score to nodes, edges, sub-graphs, and graphs at every observation, and  $\text{Sim}\{\cdot, \cdot\}$  at the coarse level. The larger  $f(\cdot)$  is, the higher the probability of the graph object being abnormal.

We introduce G-TSAD methods capable of detecting individual graph-based anomaly types or their combinations throughout Section VI. Note that although five types of graph anomalies are observed in the example shown in Fig. 3, currently, there is no study that targets detecting all types of anomalies. None of the existing methods target to detect anomalous  $\text{Sim}\{\cdot, \cdot\}$  and very few works could simultaneously detect multiple anomalous objects in graphs. This will be further discussed in Section IX.

## VI. CATEGORIZATION OF G-TSAD METHODS

Existing G-TSAD methods can be categorized according to several taxonomies. One taxonomy is based on the type of constructed graphs: static and dynamic, as discussed in Section IV-A1. A second taxonomy considers the trainable parameter perspective, i.e., for dynamic graphs, methods can be further categorized into node, edge, and  $\text{Sim}\{\cdot, \cdot\}$  learning. In the literature, no studies have explored  $\text{Sim}\{\cdot, \cdot\}$  learning. Note that

static graphs do not involve learnable parameters as node, edge and  $\text{Sim}\{\cdot, \cdot\}$  features are predefined. A third taxonomy relates to the types of anomalies that G-TSAD methods aim to detect, including node, edge, subgraph, graph-level, and  $\text{Sim}\{\cdot, \cdot\}$  anomalies. In this paper, we adopt a coarser taxonomy that encompasses all existing taxonomies mentioned above, based on the type of loss function minimized during *training*. Specifically, we group methods into four categories: AE-based, GAN-based, predictive-based, and self-supervised methods. This high-level classification is elaborated in Sections VI-A through VI-D. Within each of these categories, we further provide fine-grained classifications in Tables II–V, which summarize the type of constructed graphs (static or dynamic), learning tasks (node, edge, or  $\text{Sim}\{\cdot, \cdot\}$  learning), and the types of anomalies (node, edge, subgraph, graph, or  $\text{Sim}\{\cdot, \cdot\}$ ).

First, in AE-based methods, the reconstruction loss is the core mechanism. The model is trained to minimize the difference between the input data and its reconstruction using an encoder-decoder architecture. The goal is for the model to accurately reconstruct the normal samples, resulting in a low reconstruction error for normal data. In the test phase, anomalies are detected based on the reconstruction error.

Second, GAN-based methods operate with a more complex mechanism, consisting of a generator and a discriminator. The generator inputs both noises and real samples to reduce the randomness issue. It aims to generate fake samples that are as realistic as the real samples, so the generator loss is the reconstruction loss to minimize the difference between the fake and real samples. Meanwhile, the discriminator uses the cross-entropy loss to classify the real samples as real and the generated samples as fake. The generator and discriminator losses together provide a richer training loss, distinguishing them from pure reconstruction-based AE methods. In the test phase, the losses of both components can be used as an indicator for anomaly scores.

Third, unlike the other categories that only consider the current and past data for loss calculation during training, predictive-based methods aim to forecast future values based on historical data. The loss function is the prediction error, which measures the difference between the predicted and actual future values. While this prediction error can be commonly viewed as a form of reconstruction error — since the model aims to reconstruct



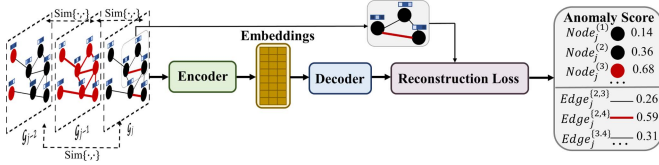


Fig. 4. Overall framework of AE-based methods. The input is a graph  $\mathbb{G}$  with three successive observations. In each  $\mathcal{G}$ ,  $m = 3$  and the edge features are not shown for simplicity.

the next time steps based on past data — the focus of predictive methods is on forecasting future values rather than reconstructing the given input data. Note that predictive-based methods, though capable of incorporating AE, GAN, or self-supervised approaches, are focused on forecasting future values rather than reconstructing input data. A sample is flagged as anomalous if the model fails to accurately predict the desired future values.

Lastly, self-supervised methods are a subgroup of unsupervised learning methods where there is no label for the training data. Compared to the three categories discussed above, self-supervised methods aim to derive more meaningful representations from the data by incorporating pretext tasks specifically designed for the unlabeled data. Their self-supervised loss functions are designed to minimize the difference between the model’s predictions and the predefined outputs associated with these pretext tasks.

Note that all method categories share common elements of deep feature learning [66], as their training involves proxy tasks that enhance representation learning. Additionally, most of the existing methods employ a notion of encoder and decoder when developing their algorithm. Furthermore, most of existing studies train on only normal data, while the test set additionally includes anomalous data to verify the methods’ detection performance. We refer to methods that only have access to normal data during training as unsupervised AD. Further details will be described in the following sub-sections. Each begins with an overview of the corresponding method category and concludes with a summary highlighting key takeaways. Accordingly, we retain the “Overview” and “Summary” labels to enhance readability.

#### A. AE-Based Methods

*Overview.* Fig. 4 and Table II illustrate the overall framework and representative methods in the AE-based category. In the literature, the AE-based methods take the full graph at every observation as the input and aim to reconstruct its components: nodes/edges features and the adjacency matrices. The origin of these methods can be traced back to vanilla AE, an encoder-decoder framework, where the encoder network  $E_\theta$  (parameterized by  $\theta$ ) learns to compress graph data into low-dimensional embeddings, and the decoder network  $D_\phi$  (parameterized by  $\phi$ ) aims to reconstruct the input. This can be formulated as:

$$\theta^*, \phi^* = \underset{\theta, \phi}{\operatorname{argmin}} \mathcal{L}_{\text{rec}}(D_\phi(E_\theta(\mathbb{G})), \mathbb{G}), \quad (2)$$

where the reconstruction loss  $\mathcal{L}_{\text{rec}}$  is often the mean squared error (MSE) or the cross entropy (CE) loss [67].

DeepSphere [68] is the first AE-based approach for G-TSAD. It incorporates an LSTM-AE with hypersphere learning to capture both intra- and inter-variable dependencies in dynamic networked graphs. It uses adjacency matrices available in these networked graphs to reconstruct normal patterns. The idea of employing AE is that AE has a strong capability in learning non-linear patterns. Such high-quality non-linear representation supports the hypersphere learning – the hypersphere boundary is trained to enclose the normal data in the representation space. The node-level objects lying outside the hypersphere tend to be anomalous. DeepSphere has proven effective in detecting anomalies in traffic networks and social media scenarios.

The main challenge of this vanilla AE framework is the non-regularized latent space i.e., no specific constraints are imposed on the latent space during training. Hence, the reconstruction process is done by sampling the data points from the non-regularized latent space. This could limit the model’s ability to reconstruct valid inputs in the test phase since AE may simply memorize to perfectly reconstruct the input data during training. Variational autoencoders (VAE) [69] come into the picture to address this issue. VAE maps the inputs to a distribution in the latent space rather than vectors as done by the vanilla AE. This means instead of learning a single encoding for each input, VAE learns a distribution for the data. VAE imposes a constraint by which the latent space has a normal distribution; this constraint guarantees that the latent space is regularized. With such a distribution, VAE allows to represent the input with some degree of uncertainty, i.e., a more generalized data representation is learned.

Recognizing the potential of VAE, many algorithms have been developed to improve G-TSAD performance. GReLeN [70] integrates VAE and a Graph Neural Network (GNN) with a graph structure learning for AD in multivariate time-series signals, collected from the server and water systems. Specifically, the latent space in VAE serves as the module for feature extraction, while GNN with the self-attention mechanism [71], which assigns the attention weights to the edge weights in the graph, captures the between-sensor dependency. Each sensor (node) is then assigned an anomaly score based on the reconstruction error. GReLeN addresses the importance of learning the inter-variable dependency between sensors by its graph-based module, as well as the intra-variable dependency learned implicitly by its VAE.

Deep Variational Graph Convolutional Recurrent Network (DVGCRN) [72], another VAE-based method, was proposed to capture both intra- and inter-variable dependencies, as well as the stochasticity present in the multivariate time-series signals, collected from the server machines and satellite monitoring systems. DVGCRN includes three components: a stacked graph convolutional recurrent network to model the multilevel intra-variable dependency; a Gaussian-distributed channel embedding module to characterize the similarity and stochasticity of different sensors by computing graph edges through the inner product of sensor embeddings; and a deep embedding-guided probabilistic generative network to model the non-deterministic inter-variable dependency in the latent space; hence it can

capture multilevel information at the different layers. Note that DVGCRN captures multi-level information since it extends the model into a multi-layer network. The reconstruction error is assigned for each sensor (node) as an anomaly score.

Although VAE can map the input to a distribution in the latent space to learn generalized representations of data, VAE may struggle to model high-dimensional distributions with complex structures. Thus, VAE may not reconstruct the samples that are as realistic and high-quality as the original data. Moreover, VAE models are sensitive to the hyperparameter choice and require careful tuning to achieve good performance. To tackle these issues, few recent studies, leveraging Normalizing Flow (NF) [73], [74], have developed and yielded superior performance as NF can model complex distributions in the latent space; hence, high-quality samples can be reconstructed. NF is a statistical method using the change-of-variable law of probabilities to transform a base distribution into a target distribution.

A recent novel NF-based method, called GANF [48] learns Directed Acyclic Graphs (DAG) inside a continuous flow model, allowing for an explicit representation of sensor dependencies in sensory systems, such as highway traffic, water systems and power grids. Specifically, in addition to learning the intra-variable dependency by an LSTM model, GANF includes a graph-based dependency encoder to capture inter-variable information in DAG. Note that DAG is a bayesian network in which a node is conditionally independent of its non-parents. NF is included to learn DAG by modeling the conditional density of each node in DAG, i.e., NF expresses the density of each node through successive conditioning on historical data and uses conditional flows to learn each conditional density. Unlike OmniAnomaly [74], which uses the reconstruction error for anomaly scores, GANF employs density estimation, identifying nodes in low-density regions of the data distribution as anomalies.

However, there are several major issues with the NF-based methods in G-TSAD. NF can be computationally expensive, especially for modeling complex distributions for large datasets since each transformation in the flow requires computing invertibility and the determinant of the Jacobian matrix. Moreover, the resulting NF models may not be highly interpretable for high-dimensional data as it is difficult to understand the relationships between the input variables and the generated samples in such complex data.

The above unsupervised methods assume that the training data must be clean. However, in real-world applications, collecting clean data poses significant challenges due to its time-consuming, costly, and labor-intensive nature. Moreover, anomalies often sneak into normal data, regarded as contaminated data, that come from the data shift or human error [77]. Consequently, existing unsupervised methods, which extensively train on normal data to model its behavior, would misdetect anomaly samples encountered during training in the test phase. To address such challenges, [76] introduced an approach, called TSAD-C. TSAD-C comprises three modules: a Decontaminator, aimed at eliminating abnormal patterns; a Long-range Variable Dependency Modeling, designed to capture long-range intra- and inter-variable dependencies, where graphs are learned through a self-attention mechanism with the attention weights assigned for the edges' weights; and Anomaly Scoring based

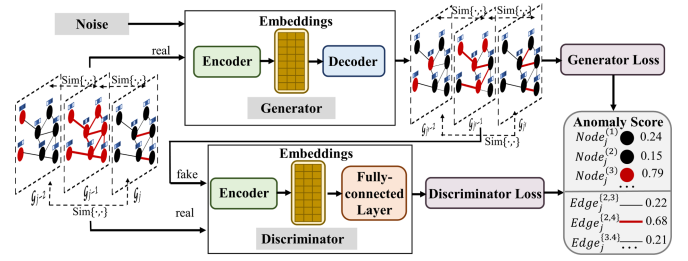


Fig. 5. Overall framework of GAN-based methods. The input is a graph  $\mathbb{G}$  with three successive observations. In each  $\mathcal{G}$ ,  $m = 3$  and the edge features are not shown for simplicity.

on reconstruction scores used to detect graph-level anomalies in both industrial and biomedical multivariate signal domains.

Note that TSAD-C employs a diffusion model - a type of generative models [78], [79], in its Decontaminator. However, similar to the drawbacks of NF, training a diffusion model can be computationally expensive, especially for large datasets. It also demands large memory resources during training when storing intermediate data representations. Moreover, diffusion models usually rely on various hyperparameters, including learning rates, batch sizes, and model architecture, which can significantly influence performance. Effectively tuning these hyperparameters can be challenging and time-consuming, requiring extensive experimentation.

*Summary.* Many studies in the AE-based methods, ranging from vanilla AE, VAE to NF, have shown promising results in G-TSAD. However, designing a model that can capture both intra- and inter-variable dependencies requires careful consideration of multiple factors. Large amounts of data and computational resources for training are required to produce satisfactory results, which can be expensive and time-consuming for many time-series applications. Also, interpretability are necessary to understand the model's behavior with the detection outcomes. This ensures that the model makes unbiased decisions on detecting anomalies. Moreover, most of the AE-based methods limit their applications to only time-series signal systems while there are many other data types such as videos, social networks, and edge streams, which are common in real-world scenarios. This restricts the adaptability and applicability of AE-based approaches to a wider variety of time-series contexts.

## B. GAN-Based Methods

*Overview.* Fig. 5 and Table III provide the concept map and representative methods in the GAN-based category. Overall, parameters of the two components of GAN, i.e., the generator and the discriminator, can be found as below:

$$\theta^*, \phi^* = \underset{\theta, \phi}{\operatorname{argmin}} \mathcal{L}_{\text{gen}}(D_{\phi}(E_{\theta}(\mathbb{G}, z)), \mathbb{G}), \quad (3)$$

$$\psi^* = \underset{\psi}{\operatorname{argmin}} \mathcal{L}_{\text{disc}}(\mathbb{D}_{\psi}(\mathbb{G}', \mathbb{G})), \quad (4)$$

where  $z$  is the random noise vector,  $\mathbb{D}$  is the discriminator network (parameterized by  $\psi$ ),  $\mathbb{G}'$  is the fake set of graph samples generated by the generator. The generator loss  $\mathcal{L}_{\text{gen}}$  is often MSE, and the discriminator loss  $\mathcal{L}_{\text{disc}}$  is often CE.



TABLE III  
SUMMARY OF REPRESENTATIVE GAN-BASED METHODS

| Method          | Year | Data Type       | Constructed Graph | Learning Task | Technique                                                | Anomaly Type | Evaluation Metric | Targeted Application                |
|-----------------|------|-----------------|-------------------|---------------|----------------------------------------------------------|--------------|-------------------|-------------------------------------|
| CCG-EDGAN [80]  | 2021 | Signals         | Static            | –             | GNN-GAN with cross-correlation graphs                    | Node         | Prec, Rec, F1     | Wind Turbine, Power Consumption     |
| HAD-MDGAT [81]  | 2022 | Signals         | Dynamic           | Node, Edge    | GNN-GAN with an attention module                         | Node         | Prec, Rec, F1     | Satellite and Soil Moisture Systems |
| STGAN [50]      | 2022 | Signals         | Static            | –             | GAN with both spatiotemporal generator and discriminator | Node         | Rec               | Traffic Data                        |
| RegraphGAN [82] | 2023 | Social Networks | Dynamic           | Edge          | Reverse GAN with edge-based substructure sampling        | Edge         | AUC               | Social Networks, Cybersecurity      |

Recent studies have applied GAN to G-TSAD. For example, [80] proposed Cross-Correlation Graph-Based Encoder–Decoder GAN (CCG-EDGAN) for unsupervised AD in multivariate signals, collected from satellite, wind turbine monitoring, and power consumption systems. CCG-EDGAN consists of three modules: a preprocessing module to transform signals into predefined correlation graphs; a generator to generate the fake data via the encoder-decoder-encoder structure, which captures the features in the correlation graphs; and a discriminator to classify the real from reconstructed data. Note that the input includes both noise and real data to help the generator improve the reconstructed samples’ diversity and avoid the mode collapse problem, which refers to the situation that the generator can only produce the fake samples that look similar to each other but are not diverse enough to represent the full range of possible samples. The anomaly score assigned for each node is computed by the reconstruction error.

CCG-EDGAN encounters two limitations. First, it could only handle the inter-variable dependency through the correlation graphs, while the intra-variable dependency could not be captured by the GAN module. Second, its anomaly score computation is only based on the reconstruction error obtained by the generator, ignoring the discriminator’s potential in detecting anomalies. To overcome the first limitation, HAD-MDGAT was proposed [81]. Unlike CCG-EDGAN, which relies on predefined correlation graphs, HAD-MDGAT uses a graph attention network to learn graphs dynamically from attention weights, allowing it to capture inter-sensor correlations. A GRU network is used to capture the intra-variable patterns. Node-level anomalies are detected by a reconstruction scheme via GAN. HAD-MDGAT also compares the performance between GAN and vanilla AE and confirms that GAN achieves a better reconstruction and could solve the overfitting problem of vanilla AE. Despite the ability to tackle the first limitation of CCG-EDGAN, HAD-MDGAT could not tackle the second issue, i.e., HAD-MDGAT only uses the anomaly scores obtained by the generator. HAD-MDGAT shows its applications in satellite monitoring and soil moisture signal systems.

Several recent methods address both limitations, i.e., capturing both intra- and inter-variable information, as well as using both the generator and discriminator for computing the anomaly score. STGAN [50] was proposed to detect node-level anomalies in the traffic networks. STGAN first constructs graphs for the traffic network, with edge weights calculated using the euclidean distance between nodes. STGAN then includes two components: a generator to model intra- and inter-variable patterns, trend,

and external features; and a discriminator to distinguish the real samples from the fake samples. STGAN calls its components, respectively, spatiotemporal generator and spatiotemporal discriminator since both are based on graph convolutional gated recurrent unit that explores the intra- and inter-variable correlations among neighboring nodes. Anomaly score is computed based on the performance of both the generator and discriminator. While the generator detects sudden changes in traffic networks, the discriminator detects traffic anomalies such as abnormal times and locations.

While HAD-MDGAT can overcome the first challenge and STGAN can tackle both challenges seen in CCG-EDGAN, the major issue of both HAD-MDGAT and STGAN methods is that, unlike CCG-EDGAN that uses both noises and the real data as the input to improve the generated samples diversity and unlike any GAN-based methods applied on images that use only noises as the input, HAD-MDGAT and STGAN set only the real data as the input and no noise is used to generate fake samples. This potentially leads to the mode collapse problem in the GAN framework.

*Summary.* Although GAN-based methods have made important contributions to the G-TSAD field, they still have some challenges that require further research. Many GAN-based methods suffer from the mode collapse problem when the generator can not produce a wide range of samples since taking only the real data as the initial input can make the generator create the same or very similar samples. Moreover, as the generator and the discriminator constantly compete against each other, the training process could be unstable and slow. Also, GAN-based methods are sensitive to the choice of hyperparameters; hence, carefully choosing them to avoid unstable training, mode collapse, and poor-quality generated samples is needed. Importantly, it is essential to design a GAN that can properly combine the detection ability of both the generator and discriminator. Lastly, to date, there are only a few GAN-based methods applied to G-TSAD, and their applicability is largely limited to time-series signal data from traffic or sensory systems. Exploring effective GAN-based methods that can be adapted to other data types, such as videos and dynamic social networks, would be a valuable direction for future research.

### C. Predictive-Based Methods

*Overview.* Fig. 6 and Table IV illustrate the overall framework and representative methods in the predictive-based category. As mentioned earlier, predictive methods are different from

TABLE IV  
SUMMARY OF REPRESENTATIVE PREDICTIVE-BASED METHODS

| Method            | Year | Data Type       | Constructed Graph | Learning Task | Technique                                                         | Anomaly Type | Evaluation Metric | Targeted Application                 |
|-------------------|------|-----------------|-------------------|---------------|-------------------------------------------------------------------|--------------|-------------------|--------------------------------------|
| SEDANSPOT [85]    | 2018 | Edge Streams    | Dynamic           | Edge          | Rate-adjusted sampling and random walked edge techniques          | Edge         | Prec, Rec, F1     | Traffic Networks, Social Media       |
| GCLNC [86]        | 2019 | Videos          | Dynamic           | Node, Edge    | A label noise cleaner with GNN and a supervised action classifier | Node         | AUC               | Real-world Videos                    |
| MTAD-GAT [83]     | 2020 | Signals         | Dynamic           | Node, Edge    | A forecasting-based module, and a reconstruction-based VAE        | Node         | Prec, Rec, F1     | Satellite Monitoring, Server Systems |
| Midas [87]        | 2020 | Edge Streams    | Dynamic           | Edge          | An online method to process each edge in constant time and memory | Edge         | AUC               | Cybersecurity, Social Media          |
| GDN [49]          | 2021 | Signals         | Dynamic           | Node, Edge    | GNN with an attention mechanism                                   | Node         | Prec, Rec, F1     | Water Systems                        |
| GTA [88]          | 2021 | Signals         | Dynamic           | Node, Edge    | Gumbel-softmax sampling for connection learning policy            | Graph        | Prec, Rec, F1     | Water Systems, Satellite Monitoring  |
| Eland [89]        | 2021 | Social Networks | Dynamic           | Edge          | An action predictor with sequence augmentation techniques         | Graph        | Prec, AUC         | Social Networks                      |
| Series2Graph [90] | 2022 | Signals         | Dynamic           | Node, Edge    | A two-embedding space for the densest regions in graphs           | Sub-graph    | Accuracy          | Various Applications                 |
| WAGCN [51]        | 2022 | Videos          | Dynamic           | Node, Edge    | GNN with a graph construction module                              | Graph        | AUC               | Real-world Videos                    |
| FuSAGNet [84]     | 2022 | Signals         | Dynamic           | Node, Edge    | Sparse-AE and GNN with an attention mechanism                     | Node         | Prec, Rec, F1     | Water Systems, System Security       |
| MST-GAT [91]      | 2023 | Signals         | Dynamic           | Node, Edge    | A multimodal GNN with a graph attention module                    | Node         | Prec, Rec, F1     | Various Industrial Applications      |
| GiCISAD [92]      | 2025 | Videos          | Dynamic           | Node, Edge    | Diffusion models with a GNN-based forecasting module              | Graph        | AUC               | Human Activity Recognition           |

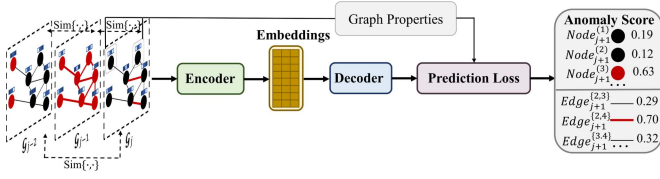


Fig. 6. Overall framework of Predictive-based methods. The input is a graph  $\mathbb{G}$  with three successive observations. In each  $\mathcal{G}$ ,  $m = 3$  and the edge features are not shown for simplicity.

other categories as they aim to predict future values based on current and past values. A sample is detected as an anomaly if the model can not provide accurate time-series forecasting. Specifically, at each observation  $j$ , a predictive-based approach calculates an expected graph  $\bar{\mathcal{G}}_{j+1}$ . The anomaly score is then computed as the difference between the expected graph  $\bar{\mathcal{G}}_{j+1}$  and the actual graph  $\mathcal{G}_{j+1}$ . The time-series prediction problem is difficult, as learning the long-distance intra-variable complexity of observations is necessary. As a result, in addition to the features of nodes/edges, and the connectivity patterns between nodes represented by the adjacency matrices, other important properties, such as the node degree, shall be considered – this enables better capturing of the long-distance dependencies. This framework can be defined as follows:

$$\theta^*, \phi^* = \underset{\theta, \phi}{\operatorname{argmin}} \mathcal{L}_{\text{pred}}(D_{\phi}(E_{\theta}(\mathbb{G})), \mathcal{G}_{j+1}), \quad (5)$$

where the prediction loss  $\mathcal{L}_{\text{pred}}$  is often MSE, Root MSE (RMSE), or CE. The first term in the loss represents  $\bar{\mathcal{G}}_{j+1}$ .

Many recent studies have shown the potential of GNN with a graph attention network in detecting anomalies in the future. For example, [49] proposed Graph Attention-based Forecasting (GDN) to dynamically learn the graph by an attention mechanism with attention weights assigned for edge weights, then predict the behavior of each sensor at each time step by

learning the past. This helps detect sensor (node) anomalies that deviate from their expected behavior. An anomaly score for each sensor at each time step is computed using the absolute error value. Normalization is then applied to all anomaly scores across all time steps for each sensor. Importantly, the obtained attention weights are used to provide explainability for the detected anomalies. GDN addresses very important issues in G-TSAD research, i.e. different sensors may have different behaviors and properties (e.g., some sensors measure the water pressure while others measure the flow rate). However, existing GNN-based methods use the same set of model parameters for all sensors. Moreover, many of them pre-define the graph nodes/edges/adjacency matrices. Though using the pre-defined matrices results in a more stable training phase, defining the matrices themselves requires prior knowledge about the dataset, which may not be reachable in many applications. In spite of the GDN's great capability to address real-world issues, its limitation is that it detects anomalies mainly based on inter-variable dependencies; the intra-variable relations across different time steps are ignored.

To overcome the limitations of GDN, MTAD-GAT [83] was proposed. It leverages the graph attention network for learning the graphs and capturing both inter-, and intra-variable dependencies. The key ingredient in MTAD-GAT is that there are two graph attention layers with respect to the inter- and intra-variable information: the feature-oriented graph attention layer to capture the relations between sensors and the time-oriented graph attention layer to capture the dependency among observations. MTAD-GAT jointly trains a forecasting-based module, for single-observation prediction with RMSE, and a reconstruction-based VAE module, for capturing the data distribution of the entire time series. Although MTAD-GAT could address the issue in GDN studies, its graph attention module could not provide interpretability on the characteristics of different sensor types, present in the multivariate sensory systems, as done by other GDN works [49], [84].

As seen from the above studies, a graph structure learning strategy (i.e., learning the features of nodes/edges and adjacency matrices) via attention mechanisms is important and commonly used in many time-series applications. This is because, with limited prior knowledge, it is difficult to pre-define the graph structure that can carry comprehensive information about the graph. Meanwhile, uncovering the important patterns in graphs is possible through learning with GNN. However, many existing methods learn the adjacency matrices in the sensor embedding space by defining measurement metrics such as the cosine similarity or top-k nearest neighbors (kNN) [93]. Cosine similarity, which computes the dot product of all sensor embeddings, results in a fully connected graph, leading to quadratic time and space complexity with respect to the number of sensors. top-kNN is another approach based on a similarity metric, such as cosine similarity, Gaussian similarity, or euclidean distance, that selects only the top-k neighboring nodes, which can not entirely underline the strong and wide-spread connections among sensor embeddings.

To overcome this challenge, GTA [88] presents the connection learning policy based on the Gumbel-softmax sampling approach. This is to learn bi-directed edges between sensors, thereby avoiding the issue of selecting node neighbors by the top-kNN method. GTA also handles the intra-variable dependency from the graph sequence by a transformer-based graph convolution network. Moreover, to tackle the quadratic complexity, GTA proposes a multi-branch attention mechanism to substitute the original-head self-attention method. A forecasting-based strategy is then adopted to predict the graph at the next time step and return an anomaly score assigned for each graph by MSE. GTA demonstrates its effectiveness in multivariate sensory systems, such as water and satellite monitoring systems.

In contrast to detecting abnormal sensors (nodes) of the graph in many multivariate signal systems, several studies show an interest in detecting abnormal edges (e.g., the unexpected interactions between sensors or the anomalous edges in an edge stream). SEDANSPOT [85] considers an AD problem in an edge stream, where the edge anomalies tend to connect parts of the graph that are sparsely connected or occur as bursts of activity. An example of a burst is an occasion (e.g., a burst of longer-than-usual phone calls during festivals). SEDANSPOT exploits these abnormal behaviors by proposing a rate-adjusted sampling technique that downsamples edges from bursty periods of time and employs a holistic random walk-based edge anomaly scoring function to compare incoming edges and the whole graph. This approach is applied in traffic networks, co-authorship analysis, and social media scenarios.

Midas [87] also tackles the same problem in detecting anomalous edges in an edge stream, particularly for cybersecurity and social media applications. However, unlike SEDANSPOT, which identifies individual abnormal edges, Midas aims to detect microcluster anomalies, which are the sudden appearing bursts of activity sharing many repeated edges. Midas provides explanations on the false positive rate and is also an online method that processes each edge in constant time and memory, while SEDANSPOT does not address these issues. Moreover, SEDANSPOT and Midas propose different hypotheses

on anomaly scores. SEDANSPOT designs an anomaly scoring function that gives a higher score to an edge if adding it to a sample of edges produces a large change in distance between its incident nodes. Midas computes the Gaussian likelihood of the number of edge occurrences in the current timestamp and declares an anomaly if the likelihood is below an adjustable threshold, which is to guarantee low false positive probability.

Not limited to node and edge AD in the context of time-series data, several predictive-based studies show the interesting task of detecting anomalous sub-graphs. As mentioned in Section V, detecting abnormal sub-graphs is much more difficult than abnormal nodes and edges since abnormal sub-graphs vary in size and inner structures. It is even more difficult when individual nodes/edges are normal but abnormal when considered as a group of nodes/edges.

Series2Graph [90] tackles these problems by proposing a novel idea for detecting anomalous sub-graphs with varying sizes in the graphs, which are constructed from multivariate time-series signals, collected from both industrial and biomedical domains. Specifically, Series2Graph first extracts sub-sequences (aka time intervals) from the time series using a sliding window technique. It then projects the sub-sequences onto a two-dimensional embedding space. The graph is constructed by considering the densest regions in the two-dimensional embedding space, where a notable concentration of projected sub-sequences is observed. These dense regions are selected as nodes, which serve as summaries of the repetitive patterns observed in the data, indicating normal behavior. To create the edges, Series2Graph determines the edge weights based on the frequency of occurrences of one sequence immediately following another in the input time series. If there is a transition between two nodes, they are connected by an edge; otherwise, no edge is created. Series2Graph determines the abnormality level of the sub-graphs using the node degree (aka the number of connected edges to the node) and the edge weights (aka the number of consecutive occurrences of two sequences).

Existing predictive-based methods have shown the capability to deal with many time-series applications, ranging from time-series signals, edge streams to social networks. However, none of these approaches are applicable to videos in which a video is a sequence of image frames indexed in time. GCLNC [86], WAGCN [51], and WSANV [94] are some of the earliest G-TSAD studies working on real-world surveillance videos. While most of the existing studies use only normal data during their training phase, these three studies have taken advantage of weakly-supervised labels (e.g., include noisy labels as wrong annotations of normal graphs) during training to improve the detection of abnormal graph-level frames in videos. As a video sequence has temporal evolution between frames, these methods construct graphs, where each frame is modeled as a graph node, and the adjacency matrices are computed based on not only the similarity metric of the inter-variable features but also the intra-variable proximity, i.e., the short-distance intra-variable dependency to capture relationships among frames. Note that instead of pre-defining the adjacency matrices, they adjust them as the model is trained. A computed anomaly score based on CE is then assigned for each frame.



TABLE V  
SUMMARY OF REPRESENTATIVE SELF-SUPERVISED METHODS

| Method        | Year | Data Type       | Constructed Graph | Learning Task | Technique                                                                  | Anomaly Type    | Evaluation Metric | Targeted Application              |
|---------------|------|-----------------|-------------------|---------------|----------------------------------------------------------------------------|-----------------|-------------------|-----------------------------------|
| AddGraph [99] | 2019 | Social Networks | Dynamic           | Edge          | Temporal GNN and attention-based GRU                                       | Edge            | AUC               | Social Networks                   |
| TADDY [100]   | 2021 | Social Networks | Dynamic           | Edge          | A graph transformer with an edge-based substructure sampling               | Edge            | AUC               | Social Networks, Internet Systems |
| CRFAD [101]   | 2021 | Videos          | Dynamic           | Node, Edge    | A self-attention module and conditional random fields                      | Graph           | AUC               | Surveillance Videos               |
| CAAD [102]    | 2021 | Videos          | Dynamic           | Node, Edge    | A contrastive attention module and a classification module                 | Graph           | AUC               | Real-world Videos                 |
| EEG-CGS [52]  | 2023 | Signals         | Static            | –             | GNN-based AE and CL modules, with a sub-graph sampling technique           | Node, Sub-graph | Prec, Rec, F1     | Seizure Analysis                  |
| mVSG-VFP [53] | 2023 | Signals         | Static            | –             | GNN-based AE and CL modules with a predefined $\text{Sim}\{\cdot, \cdot\}$ | Graph           | Prec, Rec, F1     | Vehicle Failure Prediction        |
| IVAD [103]    | 2023 | Videos          | Dynamic           | Node, Edge    | A dual-monitoring approach for objects and their interactions              | Edge, Graph     | AUC               | Real-world Videos                 |
| STGA [104]    | 2023 | Videos          | Dynamic           | Node, Edge    | A multi-head attention GNN                                                 | Graph           | AUC               | Real-world Videos                 |

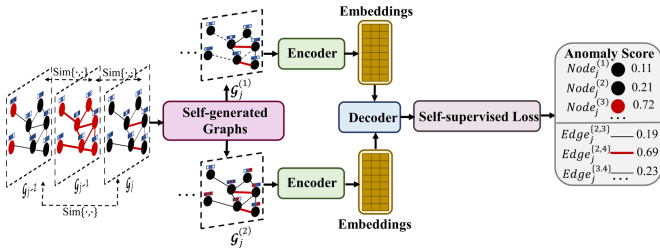


Fig. 7. Overall framework of Self-supervised methods. The input is a graph  $\mathbb{G}$  with three successive observations. In each  $\mathcal{G}$ ,  $m = 3$  and the edge features are not shown for simplicity.

Despite the capability of these methods on videos, they only consider frame-level graphs, hence, the relationships between frames are captured. However, in practice, it is essential to model a video as a stream of time-evolving object-level graphs, where a node is an object and edges represent the relationships among objects. Object-level graphs can help to determine the abnormal changes in actions/movements of an object across frames in a video.

*Summary.* Predictive-based methods have shown great potential in various time-series applications, from time-series signals, edge streams, social networks to videos. Interestingly, many predictive-based methods integrate the reconstruction and prediction tasks to yield more accurate anomaly scores. However, several challenges still remain unsolved. While existing methods have shown their effectiveness in building graphs to capture inter-variable information, one of the biggest challenges of these methods is modeling the long-term intra-variable dependency, which is required to effectively perform time-series forecasting tasks [95]. For example, existing methods only consider the intra-variable dependency in a short-term period where observations close to each other in the time series are expected to be similar. However, as mentioned in Section III, trend, seasonality, and unpredictability patterns are, in nature, always available in time-series data. Hence, forecasting future values simply based on near-distance past values is problematic. Plus, a large amount of data with a diversity of patterns is required during training for the forecasting tasks, but in practice, having access to a comprehensive dataset is very difficult in many time-series

applications. For example, in medical systems, it is difficult to obtain sufficient patient data that covers all types of diseases and rare conditions due to privacy concerns and limited access to patients over time.

#### D. Self-Supervised Methods

*Overview.* Fig. 7 and Table V provide a visualization of the overall framework and representative methods in the self-supervised category. Self-supervised learning (SSL), a subset of unsupervised methods [96], has provided novel insights into training without annotated labels [97]. Similar to other fields, SSL has demonstrated its effectiveness in graph-based learning methodologies [25]. The intuition of SSL is to learn transferable knowledge from massive unlabeled data with well-designed pretext tasks and then generalize the learned knowledge to the downstream tasks [98]. In the SSL-based G-TSAD research, a sub-group of SSL methods, called contrastive learning (CL), has been used so far. CL aims to learn useful representations of data by contrasting positive and negative pairs, i.e., CL is trained to create an embedding space that pulls positive (similar) samples close together and pushes negative (dissimilar) samples far apart. To achieve this, a contrastive loss function is defined to encourage the model to maximize the similarity between positive samples while minimizing the similarity between negative samples. This approach can be formally described as:

$$\theta^*, \phi^* = \underset{\theta, \phi}{\operatorname{argmin}} \mathcal{L}_{\text{con}} \left( D_{\phi} \left( E_{\theta}(\mathbb{G}^{(1)}), E_{\theta}(\mathbb{G}^{(2)}) \right) \right), \quad (6)$$

where  $\mathcal{L}_{\text{con}}$  is the contrastive loss,  $\mathbb{G}^{(1)}$  and  $\mathbb{G}^{(2)}$  are two different sets of graph samples,  $D_{\phi}$  is the discriminator that estimates the similarity between these two sets of graph samples. In unsupervised AD, data augmentation is essential to create negative samples/pairs while every two normal samples can potentially form a positive pair. Applying augmentation on graphs is challenging compared to other data types; this will be discussed in Section IX.

AddGraph [99] is an unsupervised anomalous edge detection technique that has access to only normal edges during training. It employs CL to train its graphs obtained from social networks, where the positive samples are all the available normal edges

while negative samples are generated by applying the Bernoulli distribution to the normal edges. AddGraph employs a temporal graph convolutional network with an attention-based GRU to learn long and short-term temporal patterns of the nodes. The hidden state of the nodes at each time step is used to compute the anomaly scores for all edges. As the generated negative edges may have a possibility of being normal, it is inappropriate to use a strict loss function such as CE; hence, AddGraph proposes a margin-based pairwise loss to distinguishes the positive edges from the generated negative ones.

Another CL study aiming to detect abnormal edges in social networks and cyber security systems, called TADDY [100], also assumes that all normal edges are positive during training, similar to AddGraph. However, TADDY proposes a different negative sampling strategy. Given positive pairs, TADDY randomly generates candidate negative pairs with a number equal to the number of positive pairs. It then checks these candidates to ensure that they are different from the existing normal edge set. Then, TADDY develops a framework with four components: an edge-based substructure sampling to sample the target nodes and the surrounding nodes; a node encoding component to generate the node embeddings in both intra- and inter-variable spaces; a graph transformer to extract the intra- and inter-variable knowledge of edges; and a discriminative anomaly detector based on CE to calculate anomaly scores for all edges.

Although TADDY demonstrates its effectiveness on six real-world graph datasets, while AddGraph uses only two social network datasets, TADDY still remains several issues. First, TADDY implements a random selection technique for generating negative samples, but it does not ensure that they do not belong to the normal set. Hence, its negative sampling approach is less reliable than AddGraph. Second, given the random selection technique of TADDY, the bias can easily occur when the negative sample set is not sufficient to represent the full distribution of negative pairs; hence, additional sampling techniques are required. Lastly, it employs a CE loss to distinguish normal edges from negative edges; however, as denoted in AddGraph, since it is not guaranteed that all generated negative samples are actual non-normal, using a strict loss (e.g., CE) is inappropriate.

The major challenge of both AddGraph and TADDY is that they consider the generated negative samples as actual abnormal samples since there is no access to the actual abnormal data during training, and the generated negative samples are created based on the given normal data. This limits the methods' performance as the generated negative samples might be just a variation of the normal data or a subset of the possible abnormal distribution, resulting in biased representation learning for the actual abnormal data.

Very recent studies [52], [53] have addressed these challenges by recognizing that the negative samples generated based on the normal data may differ from the actual abnormal data that will be observed in the test phase. These methods demonstrate that discriminating the positive and negative pairs during training through minimizing the contrastive loss is just for providing a less noisy and more reliable representation space for the *normal* data as all the negative samples are created from normal data – in other words, the generated negative samples are

just a transformation of normal data, not an actual abnormal sample.

EEG-CGS [52] introduces a sub-graph sampling technique for the epilepsy application of detecting abnormal channels and regions in the brain using multivariate time-series signals. EEG-CGS first represents the signals as static graphs, constructed based on the euclidean distance or correlation matrices, to capture the relations between sensors. It then proposes a local sub-graph sampling strategy to sample contrastive pairs. For each target node, a positive sub-graph is sampled by closely connected nodes based on controlling the radius of surrounding nodes, while a negative sub-graph is sampled by finding the farthest nodes using the same radius. By sampling positive and negative pairs based on the knowledge of local sub-graphs, EEG-CGS overcomes the issues of the random selection technique seen in TADDY.

Similar to EEG-CGS, mVSG-VFP [53] incorporates the reconstruction and contrastive modules to detect vehicle failures. However, its module design differs from EEG-CGS. mVSG-VFP first uses a sliding window to generate segments in the time-series vehicle data and then builds mini-batches of segments. Each segment is represented as a static graph, where each sensor is a node, and edges are predefined using Mutual Information between sensor pairs, measuring the statistical dependence between two sensors. Regarding the reconstruction module, mVSG-VFP masks a segment of one sensor at a time and encourages the GNN-AE module to reconstruct it. Regarding the contrastive module, unlike other CL methods that positive and negative samples are created through a contrastive pair sampling strategy, mVSG-VFP leverages a unique characteristic in their vehicle data, i.e., time-series segments recorded during a trip (aka a continuous recording of sensors) are more similar compared to those that are produced during another trip. This acts as a predefined  $\text{Sim}\{\cdot, \cdot\}$ , providing an additional layer of insight. While not learnable, this  $\text{Sim}\{\cdot, \cdot\}$  operates at the segment-label level by utilizing prior knowledge about whether segments belong to the same vehicle trip.

Given this unique property, adjacent segments in the mini-batch are considered as positive samples, and the remaining segments in the same mini-batch are denoted as negative samples. Then, a contrastive loss is applied to pull adjacent segments closer and push them away from the rest of the mini-batch. mVSG-VFP uses the reconstruction error to detect anomalous graph-level segments. Due to the data's property, a detected abnormal segment can imply that its adjacent segments are also anomalies. As such, mVSG-VFP addresses both intra- and inter-variable dependencies, while EEG-CGS only considers the latter. mVSG-VFP also demonstrates superior performance by leveraging  $\text{Sim}\{\cdot, \cdot\}$ , compared to other graph-based methods like GDN that do not utilize it. The main challenge of mVSG-VFP is that the way of generating contrastive pairs is only applicable to a data compound of operational blocks like vehicle trips, a property that is not available for many applications where using the knowledge of adjacent segments is meaningless.

CL has also been successfully applied for detecting anomalies in videos. Many existing studies such as CFRAD [101] overlook the important matter of developing models with interpretability

that can interpret the causes of anomalies. An important application of video AD is to understand and take appropriate actions whenever an anomaly occurs. A response to an abnormal event in a video is usually dependent on its severity, which cannot be easily understandable without having an interpretable model. To address this, IVAD [103] proposes a dual-monitoring approach that monitors individual objects (aka local object monitoring), and their interactions with other objects present in the scene (aka global object monitoring) in surveillance videos.

In the local branch, each object is monitored independently to determine if its action (e.g., body movement) is anomalous. An anomaly score for each frame, based on reconstruction loss, is computed by a GRU-AE. In the global branch, the relationships between objects (e.g., a human, a predicate, and a bike) within a frame are monitored. This creates a scene graph where objects and predicates (i.e., words that relate two objects) are respectively defined as nodes and edges. CL is adopted in this global branch. Positive pairs are selected from the normal training set, while negative pairs are randomly generated and verified to ensure that they do not belong to the positive set. A contrastive loss is employed to minimize the euclidean distance between positive samples and maximize the distance between positive and negative samples. This loss is also used as an anomaly score for each object in a frame. Finally, an in-depth analysis on both branches is performed to interpret the root cause of anomalies. If the cause of an anomaly is from the global branch, it is interpreted that the edge-level anomaly is caused due to a previously unseen relationship between objects. Otherwise, if the source of an anomaly is from the local branch, it is caused due to an anomalous object action in the graph.

*Summary.* Existing SSL studies have shown interesting ideas and addressed some problems seen in the other three categories of G-TSAD methods. For example, SSL methods incorporate a variety of pretext tasks, such as CL and masked reconstruction, to enhance the representation of normal data. These pretext tasks allow SSL to better learn complex patterns, leading to improved AD performance. Additionally, unlike predictive methods, which focus on modeling each frame in video data as a graph node, SSL methods can go beyond this limitation by constructing object-level graphs. This allows SSL methods to capture detailed interactions between objects within the data. However, as SSL is a new topic to G-TSAD, it has not yet confirmed its effectiveness in a variety of applications compared to predictive methods. Moreover, it is suggested that different effective graph augmentations for positive and negative sample generation be employed to improve performance.

In conclusion, while there may be some overlap in the deep feature learning mechanisms and the use of loss functions, such as reconstruction losses, shared across all method categories, the key distinction lies in the specific losses each category minimizes during training. AE-based methods focus purely on reconstruction error of the previous or current input data. GAN-based methods minimize both generator and discriminator losses. Predictive-based methods minimize prediction errors rather than merely reconstructing the input data, and self-supervised methods minimize pretext-based losses.

## VII. DATASETS AND APPLICATIONS

In this section, we present commonly used datasets related to time-series signals, social networks, and videos that have been widely adopted in G-TSAD research. Table VI summarizes these datasets based on their type, characteristics, specific anomalies, and application domains.

### A. Time-Series Signals

Numerous existing G-TSAD studies have trained models using popular benchmark time-series signal datasets, such as Yahoo S5 [105], NASA [106], SWaT [107], WADI [108], SMAP [74], and SMD [74]. However, it has been highlighted in recent literature that these benchmarks possess various flaws, including mislabeled ground truth, triviality, unrealistic anomaly density, and run-to-failure bias [8], [126], [127]. This renders them unsuitable for the robust evaluation and comparison of G-TSAD algorithms. Therefore, improving benchmarking practices is crucial to ensure a more accurate assessment of G-TSAD methods. [8] has introduced The UCR Time Series Anomaly Archive, designed for evaluating and benchmarking AD algorithms. [109] has also introduced The UCI Machine Learning Repository, which contains datasets that cover a broad range of domains such as finance, healthcare, biology, and social sciences. Additionally, [110] has recently introduced the voraus-AD dataset for robotic applications. These datasets serve as valuable resources for testing and developing TSAD algorithms. Moreover, recent studies [76], [128], [129] have showcased the superiority of their proposed TSAD methods on several reliable alternatives collected from biomedical domains such as The PhysioNet [111], The European ST-T database [112], PTB-XL [113], ICBEB [114], and TUSZ [115].

### B. Social Networks

As clarified in Section IV-B and Fig. 2(a), social networks, which are represented as dynamic graphs, are also considered as time-series data. This is because there are always fresh persons who enroll in the community network every day/month/year, hence, the relationship between individuals is changing over-time. Detecting anomalies in these dynamic social networks gains significant attention in G-TSAD research. Various datasets on social networks have been used. For instance, UCI Messages [116] is a social network dataset constructed into dynamic graphs, where each node represents a user, and each edge indicates a message exchange between two users. Email-DNC [117] is network of emails, where each node indicates a person in the United States Democratic Party, and each edge denotes an email sent from one person to another. Digg [118] is a network dataset collected from a news website, where each node represents a website user, and each edge indicates that one user replies to another user. Bitcoin-Alpha and Bitcoin-OTC [119], [120] are two “who-trusts-whom” networks of bitcoin users engaged in trading on the Internet platforms, where each node denotes a user, and an edge is present when one user rates another on the platform. AS-Topology [121] is a network connection



TABLE VI  
SUMMARY OF COMMONLY USED DATASETS IN G-TSAD RESEARCH

| Category            | Dataset                 | #Features             | Anomalies                                      | Application                            |
|---------------------|-------------------------|-----------------------|------------------------------------------------|----------------------------------------|
| Time-series Signals | Yahoo S5 [105]          | 8                     | Spikes, trend changes in traffic and energy    | Real-world scenario monitoring         |
|                     | NASA [106]              | 25                    | Sensor failures, predictive maintenance faults | Machine predictive maintenance         |
|                     | SwaT [107]              | 51                    | Operational faults in industrial control       | Water infrastructure monitoring        |
|                     | WADI [108]              | 127                   | Leaks, flow anomalies in water distribution    | Water distribution monitoring          |
|                     | SMAP [74]               | 55                    | Unexpected soil moisture readings              | Soil moisture monitoring               |
|                     | SMD [74]                | 38                    | Memory leaks, unexpected server downtimes      | Internet server monitoring             |
|                     | UCR Archive [8]         | varied                | Various anomalies across multiple domains      | AD in multiple fields                  |
|                     | UCI Repository [109]    | varied                | Broach range of domain-specific anomalies      | AD in various domains                  |
|                     | voraus-AD [110]         | 20                    | Pick-and-place operational faults              | Robotics and industrial automation     |
|                     | PhysioNet [111]         | 15                    | Cariac and physiological signal anomalies      | Physiological monitoring               |
|                     | European Database [112] | 12                    | Abnormal ECG waveforms in cardiac conditions   | Cardiac abnormality detection          |
|                     | PTB-XL [113]            | 100                   | ECG abnormalities from diverse patient groups  | ECG-based heart disease diagnosis      |
|                     | ICBEB [114]             | varied                | ECG data with abnormal cardiac disorders       | Detection of various cardiac disorders |
|                     | TUSZ [115]              | 20                    | EEG signals with normal and seizure activity   | Seizure detection in epilepsy patients |
| Social Networks     | UCI Messages [116]      | 1899 nodes, 20K edges | Unusual message exchanges between users        | Messaging graph-based networks         |
|                     | Email-DNC [117]         | 1869 nodes, 24K edges | Irregular emails in political networks         | Email network security and AD          |
|                     | Digg [118]              | 30K nodes, 85K edges  | Unusual reply behaviors in a social news       | User interactions on social platforms  |
|                     | Bitcoin-Alpha [119]     | 3.7K nodes, 24K edges | Trust rating anomalies in Bitcoin trading      | Fraud detection in Bitcoin             |
|                     | Bitcoin-OTC [120]       | 5K nodes, 35K edges   | Unusual trust relationships among Bitcoin      | Frauds in cryptocurrency networks      |
|                     | AS-Topology [121]       | 45K nodes, 110K edges | Irregular patterns in Internet systems         | AD in Internet infrastructure          |
| Videos              | UCF-Crime [122]         | 1900 videos           | Violent activities in surveillance videos      | Surveillance AD in security scenarios  |
|                     | Xd-Violence [123]       | 4754 videos           | Violent events recorded in diverse scenarios   | Violence AD in surveillance footage    |
|                     | ShanghaiTech [124]      | 437 videos            | Unusual pedestrian movements                   | Behavior analysis in outdoor spaces    |
|                     | UCSD-Peds [125]         | 98 videos             | Unusual events on pedestrian walkways          | Pedestrian safety and behavior         |

dataset collected from autonomous Internet systems, where each node denotes an autonomous system, and each edge indicates a connection between two autonomous systems.

### C. Videos

As illustrated in Section IV-B and Fig. 2(b), video data is considered as the time series as it is a sequence of frames captured over time. The frames are arranged sequentially to create the illusion of motion when played back at a rapid pace. This sequential nature of video data, where each frame depends on the preceding and succeeding frames, makes it inherently a time-series data. Various video datasets have been used in G-TSAD research such as UCF-Crime [122], Xd-Violence [123], ShanghaiTech [124], and UCSD-Peds [125]. Specifically, UCF-Crime is a large-scale dataset that consists of 13 types of anomalies captured by CCTV camera indoors and outdoors during day and night scenarios. Xd-Violence is a large-scale dataset composed of violent videos from diverse scenarios recorded from CCTV cameras. ShanghaiTech is a medium-scale dataset recorded by a CCTV camera in an outdoor location. UCSD-Peds is a small-scale dataset acquired by a camera to record pedestrian walkways.

## VIII. EVALUATION METRICS

Evaluation metrics play a crucial role in assessing the performance of G-TSAD algorithms. This section presents the commonly used evaluation metrics, and discusses several issues of evaluation protocols and potential solutions.

In the domain of time series signals, several metrics such as F1 score (F1), Precision (Pre), and Recall (Rec) are commonly used. However, it is important to highlight that recent critiques [9] have raised concerns about the evaluation protocol employed in many studies [20], [130], [131]. Specifically, these existing studies compute F1 score after applying a peculiar evaluation protocol, called point adjustment (PA). Generally, PA considers that if at least one point within an anomaly segment is detected as

an anomaly, the entire segment is then considered to be correctly detected as an anomaly. Consequently, this practice leads to high F1 scores, yet it results in a high possibility of overestimating the model performance. [9] conducted experiments demonstrating that without PA, the performance of existing methods yields no significant improvement over the baseline. Note that the baseline is simply a randomly initialized reconstruction model, such as an untrained autoencoder that consists of a single-layer LSTM. To address this issue, [9] proposed a new evaluation protocol, named PA%K, where K denotes the threshold, to mitigate the overestimation effect of PA. The concept behind PA%K is to apply PA to a segment only if the ratio of the number of correctly detected anomalies within that segment to its length exceeds the threshold K. Note that K can be selected manually based on prior knowledge.

While PA%K can address the PA issue, selecting a decision threshold remains non-trivial in TSAD research. There are four scenarios for threshold finding. The first scenario is a common practice where the threshold is selected directly on the *labeled* test set [72], [132], [133], which can lead to biased and unfair evaluations. In the second scenario, the threshold is selected based on a *labeled* validation set that yields the best F1 score. The selected threshold is then applied to the test set [49]. However, the first two cases may not align with real-world unsupervised applications where labeled data is unavailable. To tackle this limitation, the third scenario advocates for using an *unlabeled* validation set to select the threshold, which is then applied to the test set [76].

The issue of the above threshold-dependent scenarios lies in their evaluation of the model's performance at a single threshold, which may not capture the model's behavior across all possible thresholds. Hence, several methods in the domains of time-series signals, as well as social networks and videos [48], [51], [99], [100], [101], [103], [128] have adopted the fourth scenario, employing the *threshold-independent* metrics such as Area Under the Receiver-Operating Characteristic Curve (AUC) [134],

[135], and Area Under the Precision-Recall Curve (APR) [136]. This offers a comprehensive assessment of the model's performance by considering multiple thresholds, which allows for a deeper understanding of the trade-offs between Prec and Rec, facilitating better decision-making for a given application.

Overall, depending on the specific applications at hand and the goals of the evaluation, it is essential to use an appropriate method for selecting threshold as different scenarios may necessitate different approaches, such as supervised, semi-supervised or unsupervised approaches, to ensure accurate and fair evaluations. Moreover, it is advisable to employ a combination of evaluation metrics to comprehensively assess the model's performance.

## IX. CHALLENGES AND FUTURE DIRECTIONS

Although G-TSAD is a new topic, it has been rapidly growing in popularity, as can be inferred by the number of studies published in the prestigious Artificial Intelligence venues described in Section VI. However, detecting anomalies in graphs with evolving graph features and adjacency matrices is challenging, leading to several major concerns in the existing studies. In this section, we analyze these challenges and pinpoint potential directions for future studies.

*Theoretical Foundation and Explainability.* Despite its great success in various time-series applications, most existing G-TSAD methods are designed and evaluated by empirical experiments without sufficient theoretical foundations to prove their reliability. Plus, they ignore the explainability of learned representations and predicted results – e.g., what are the important features of nodes and edges, and adjacency matrices in graphs? Is the output (e.g., anomaly score) sufficient to conclude the abnormality of graph objects? These are important concerns for interpreting the model behavior. Therefore, we believe that in addition to the empirical design, providing a solid theoretical foundation and a deep analysis of the learned representations to improve the model's generalization and robustness are crucial.

*On Equivalence between Intra-variable and Inter-variable Dependencies.* Capturing both intra- and inter-variable dependencies in time-series data is very important for effective AD. However, many existing methods could not handle this problem. Regarding intra-variable dependency, time-series data often involve long-term intra-variable correlations, yet many existing studies could not tackle this issue. Regarding inter-variable dependency, it is difficult to pre-define the graph features and adjacency matrices for many time-series systems due to the limited prior knowledge of the developer. Recent studies have shown that without pre-defined graphs, graph structure learning techniques [93] can learn the graph information, i.e., the node/edge features and adjacency matrices, which dynamically evolves over time. This provides a better capturing of the system's underlying mechanism. Hence, a proper integrating of both graph learning techniques and deep sequence models is essential to provide more accurate AD.

*Graph Augmentation Strategy.* Augmenting graphs is crucial for SSL methods, e.g., generating positive and negative pairs is required in CL-based methods discussed in Section VI-D. There

are many image augmentation methods in the literature [98]. However, due to the dynamic nature of graphs (e.g., intra-variable complexity, non-euclidean structure), it is not appropriate to directly apply image-based augmentations to the graphs. Most existing SSL studies only consider random sampling techniques, sub-graph sampling, or graph diffusion for augmentation – this may provide limited diversity and uncertain invariance. Having additional yet effective augmentation techniques such as feature-based (e.g., masking a portion of node/edge features), edge-based (e.g., adding or removing portions of edges), sampling-based (e.g., sampling nodes and their connected edges using importance sampling or knowledge sampling), and adaptive augmentations (e.g., using learned attention schemes) [26] would be promising directions for G-TSAD studies.

*Detection of Multiple Types of Graph Anomalies.* Most of existing methods could only detect either anomalous nodes, edges, sub-graphs, or graphs. However, in many real-world systems, different observations can have different anomaly types; e.g., in some observations, only a single sensor is anomalous, but in other observations, a group of sensors is anomalous. Moreover, although few studies [53] use  $\text{Sim}\{\cdot, \cdot\}$  to facilitate detecting anomalous nodes, edges, sub-graphs, and graphs, there is no study that targets to detect anomalous  $\text{Sim}\{\cdot, \cdot\}$ . Since  $\text{Sim}\{\cdot, \cdot\}$  presents both short-term and long-term relationships across observations, analyzing  $\text{Sim}\{\cdot, \cdot\}$  can be beneficial for understanding the underlying dynamics of time-series, e.g., any anomalous  $\text{Sim}\{\cdot, \cdot\}$  can be considered as an unexpected shift of time-series dynamics. Thus, having a model that can detect multiple anomaly types, including  $\text{Sim}\{\cdot, \cdot\}$ , would be a very interesting research line.

*Broader Scope of Methodology.* Each category in the G-TSAD studies has its own strengths and drawbacks. AE-based and predictive-based methods are simple to implement as the losses are easy to build, but recovering the input and learning long-term dependencies is a challenging task to realize. Meanwhile, GAN-based methods are difficult to implement due to the adversarial training process that needs a careful selection of hyperparameters, the network architecture, and the optimization algorithm. SSL methods have shown promising results but designing the frameworks, augmentation techniques, and loss functions heavily relies on empirical experiments. These disadvantages may cause poor AD performance. To improve overall AD accuracy, leveraging the complementary strengths of these approaches is important. For example, we can integrate the AE-based and predictive-based models to detect a wider range of anomalies. While the AE-based detects anomalies that occur in the current/past time, the predictive-based model can detect anomalies that occur in the future.

Another example is a hybrid SSL method, e.g., combining the predictive SSL and contrastive SSL modules to achieve better generalization of the learned representations. Moreover, this combined approach can be adapted to different anomaly types; e.g., the contrastive SSL module learns to detect sudden spikes in the time series, while the predictive SSL module can be trained to detect future abnormal trends unseen in the training data. Overall, the hybrid SSL approach can benefit from multiple pretext tasks of the different SSL modules, yet it is important

to design an effective joint learning framework to balance each component in the model [25]. Given its great potential, hybrid methods would be interesting for further G-TSAD studies.

*Datasets and Evaluation Metrics.* As emphasized in Section VII, acknowledging the limitations of current benchmark TSAD datasets is crucial for advancing the field. It is important to invest efforts in curating high-quality benchmark datasets that reflect real-world scenarios and challenges. Moreover, as discussed in Section VIII, the existing challenges with evaluation protocols and metrics underscore the need for a comprehensive evaluation approach. It is essential to employ a combination of evaluation metrics to thoroughly assess model performance for an accurate and fair comparison between methods while improving reproducibility.

*Limitations of Graph-based Methods to Real-world Scenarios.* As discussed earlier, most existing methods assume that only normal samples are available during training. Graphs, while effective in capturing the underlying normal patterns of data, face difficulties in scenarios where this assumption does not hold. For instance, in open-set scenarios where only a limited amount of labeled anomalous data is available during training [137], graph-based methods may struggle in learning the data patterns due to insufficient abnormal samples. Recent open-set studies [128] have shown that their non-graph-based method can outperform graph-based methods [83], which assumes that the training data should contain only normal samples.

In contaminated datasets, where anomalies sneak into the normal training data and both the labels and positions of these anomalies are unknown [77], conventional graph-based methods can become ineffective. Recent studies dealing with contaminated data [76] have compared their method with state-of-the-art graph-based methods [49]. [76] showed that naively applying graph-based methods to contaminated data leads to inaccurate results, as these methods typically assume that the training data contains only normal samples and is free from noise or anomalies. In [76], the challenge of data contamination is addressed by incorporating a decontamination phase before the graph learning process.

Another limitation arises with highly complex and stochastic signals, such as brain or vehicle systems' signals, which are often noisy and unpredictable due to variabilities in brain states, environmental variabilities, drive behavior, and mechanical wear and tear on top of the common sensor and measurement noise. In such scenarios, the recent methods presented in [52], [53] suggest constructing static graphs using predefined adjacency matrices. Their experiments show the superior performance of static graphs compared to state-of-the-art dynamic graphs [49]. This improved performance could be due to the current limitations of graph learning techniques or the insufficient availability of training data to properly train these more complex methods. Further exploration of these areas remains a promising direction for future research in the field of G-TSAD.

Finally, the use of  $\text{Sim}\{\cdot, \cdot\}$  as an additional layer of supervision remains an underexplored area. To the best of our knowledge, there is only one study, i.e., [53], that leverages  $\text{Sim}\{\cdot, \cdot\}$  in this way. [53] demonstrates that incorporating  $\text{Sim}\{\cdot, \cdot\}$  can enhance model performance compared to other graph-based approaches [49] that do not make use of this measure.

However, in their approach,  $\text{Sim}\{\cdot, \cdot\}$  is predefined based on their given vehicle dataset's characteristics; the definition may not be applicable to other real-world datasets. The result of this study confirms the potentials of employing  $\text{Sim}\{\cdot, \cdot\}$  in improving AD. Developing methods that can either learn or adapt  $\text{Sim}\{\cdot, \cdot\}$  for various applications would offer new research directions for improving graph-based TSAD techniques.

## X. CONCLUSION

This paper conducts a comprehensive survey of G-TSAD. First, we introduce new concepts in G-TSAD, the major challenges of time-series data, as well as the advantage of representing them as graphs in the context of AD. Then, we present a systematic taxonomy that groups G-TSAD methods, primarily leveraging deep learning architectures, into four categories: AE-based, GAN-based, predictive-based, and self-supervised methods. For each category, we discuss the technical details of the methods, their strengths and weaknesses, as well as comparisons between them. A wide range of practical time-series applications are also introduced. Finally, we point out several limitations from both technical and application perspectives of the current research and suggest promising directions for future works. We hope this survey will serve as a useful reference for follow-up researchers to explore more research in this field.

## REFERENCES

- [1] S. Schmidl, P. Wenig, and T. Papenbrock, "Anomaly detection in time series: A comprehensive evaluation," in *Proc. VLDB Endowment*, vol. 15, no. 9, pp. 1779–1797, 2022.
- [2] A. Blázquez, A. Conde, U. Mori, and J. A. Lozano, "A review on outlier/anomaly detection in time series data," *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–33, 2021.
- [3] B. Zong et al., "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–19.
- [4] H. Ren et al., "Time-series anomaly detection service at Microsoft," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 3009–3017.
- [5] T. Kieu, B. Yang, C. Guo, and C. S. Jensen, "Outlier detection for time series with recurrent autoencoder ensembles," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 2725–2732.
- [6] K.-H. Lai, D. Zha, J. Xu, Y. Zhao, G. Wang, and X. Hu, "Revisiting time series outlier detection: Definitions and benchmarks," in *Proc. 35th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track*, 2021, pp. 1–13.
- [7] Q. Rebjock, B. Kurt, T. Januschowski, and L. Callot, "Online false discovery rate control for anomaly detection in time series," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 26487–26498.
- [8] R. Wu and E. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2421–2429, Mar. 2023.
- [9] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, "Towards a rigorous evaluation of time-series anomaly detection," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 7194–7201.
- [10] A. Deng, A. Goodge, L. Y. Ang, and B. Hooi, "CADET: Calibrated anomaly detection for mitigating hardness bias," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2002–2008.
- [11] H. Hojjati and N. Armanfard, "Self-supervised acoustic anomaly detection via contrastive learning," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2022, pp. 3253–3257.
- [12] R. Morais, V. Le, T. Tran, B. Saha, M. Mansour, and S. Venkatesh, "Learning regularity in skeleton trajectories for anomaly detection in videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11996–12004.
- [13] J.-C. Feng, F.-T. Hong, and W.-S. Zheng, "MIST: Multiple instance self-training framework for video anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 14009–14018.



- [14] M.-I. Georgescu, A. Barbalau, R. T. Ionescu, F. S. Khan, M. Popescu, and M. Shah, "Anomaly detection in video via self-supervised and multi-task learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 12742–12752.
- [15] C. Huang et al., "Self-supervised attentive generative adversarial networks for video anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 9389–9403, Nov. 2023.
- [16] N.-C. Ristea et al., "Self-supervised predictive convolutional attentive block for anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 13576–13586.
- [17] J.-C. Wu, H.-Y. Hsieh, D.-J. Chen, C.-S. Fuh, and T.-L. Liu, "Self-supervised sparse representation for video anomaly detection," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 729–745.
- [18] C. Zhang et al., "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 1409–1416.
- [19] B. Zhou, S. Liu, B. Hooi, X. Cheng, and J. Ye, "BeatGAN: Anomalous rhythm detection using adversarially generated time series," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 4433–4439.
- [20] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3395–3404.
- [21] Y. Zhang, Y. Chen, J. Wang, and Z. Pan, "Unsupervised deep anomaly detection for multi-sensor time-series signals," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2118–2132, Feb. 2023.
- [22] A. Abdulaal, Z. Liu, and T. Lancewicki, "Practical approach to asynchronous multivariate time series anomaly detection and localization," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 2485–2494.
- [23] X. Chen et al., "DAEMON: Unsupervised anomaly detection and interpretation for multivariate time series," in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2021, pp. 2225–2230.
- [24] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," 2022, *arXiv:2201.07284*.
- [25] Y. Liu et al., "Graph self-supervised learning: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 6, pp. 5879–5900, Jun. 2023.
- [26] L. Wu, H. Lin, C. Tan, Z. Gao, and S. Z. Li, "Self-supervised learning on graphs: Contrastive, generative, or predictive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 4216–4235, Apr. 2023.
- [27] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [28] Y. Xie, Z. Xu, J. Zhang, Z. Wang, and S. Ji, "Self-supervised learning of graph neural networks: A unified review," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 2412–2429, Feb. 2023.
- [29] Y. Zhou, H. Zheng, X. Huang, S. Hao, D. Li, and J. Zhao, "Graph neural networks: Taxonomy, advances, and trends," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 1, pp. 1–54, 2022.
- [30] M. M. Li, K. Huang, and M. Zitnik, "Graph representation learning in biomedicine and healthcare," *Nature Biomed. Eng.*, vol. 6, no. 12, pp. 1353–1369, 2022.
- [31] X. Liu et al., "Survey on graph neural network acceleration: An algorithmic perspective," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 5521–5529.
- [32] C. Zhang et al., "Few-shot learning on graphs: A survey," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 5662–5669.
- [33] C. Gao et al., "A survey of graph neural networks for recommender systems: Challenges, methods, and directions," *ACM Trans. Recommender Syst.*, vol. 1, no. 1, pp. 1–51, 2023.
- [34] S. Khoshraftar and A. An, "A survey on graph representation learning methods," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 1, pp. 1–55, 2024.
- [35] M. Jin et al., "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10466–10485, Dec. 2024.
- [36] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.
- [37] H. Kim, B. S. Lee, W.-Y. Shin, and S. Lim, "Graph anomaly detection with graph neural networks: Current status and challenges," *IEEE Access*, vol. 10, pp. 111820–111829, 2022.
- [38] A. D. Pazho, G. A. Noghre, A. A. Purkayastha, J. Vempati, O. Martin, and H. Tabkhi, "A survey of graph-based deep learning for anomaly detection in distributed systems," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 1, pp. 1–20, Jan. 2024.
- [39] J. Ren, F. Xia, I. Lee, A. Noori Hoshayr, and C. Aggarwal, "Graph learning for anomaly analytics: Algorithms, applications, and challenges," *ACM Trans. Intell. Syst. Technol.*, vol. 14, no. 2, pp. 1–29, 2023.
- [40] T. Bilot, N. El Madhoun, K. Al Agha, and A. Zouaoui, "Graph neural networks for intrusion detection: A survey," *IEEE Access*, vol. 11, pp. 49114–49139, 2023.
- [41] P. B. Lamichhane and W. Eberle, "Anomaly detection in graph structured data: A survey," 2024, *arXiv:2405.06172*.
- [42] O. A. Ekle and W. Eberle, "Anomaly detection in dynamic graphs: A comprehensive survey," *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 8, pp. 1–44, 2024.
- [43] A. A. Cook, G. Misirli, and Z. Fan, "Anomaly detection for IoT time-series data: A survey," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6481–6494, Jul. 2020.
- [44] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021.
- [45] G. Li and J. J. Jung, "Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges," *Inf. Fusion*, vol. 91, pp. 93–102, 2023.
- [46] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, "Deep learning for time series anomaly detection: A survey," *ACM Comput. Surv.*, vol. 57, no. 1, pp. 1–42, 2024.
- [47] F. Xia et al., "Graph learning: A survey," *IEEE Trans. Artif. Intell.*, vol. 2, no. 2, pp. 109–127, Apr. 2021.
- [48] E. Dai and J. Chen, "Graph-augmented normalizing flows for anomaly detection of multiple time series," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–16.
- [49] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4027–4035.
- [50] L. Deng, D. Lian, Z. Huang, and E. Chen, "Graph convolutional adversarial networks for spatiotemporal anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2416–2428, Jun. 2022.
- [51] C. Cao, X. Zhang, S. Zhang, P. Wang, and Y. Zhang, "Adaptive graph convolutional networks for weakly supervised anomaly detection in videos," 2022, *arXiv:2202.06503*.
- [52] T. K. K. Ho and N. Armanfard, "Self-supervised learning for anomalous channel detection in EEG graphs: Application to seizure analysis," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 7866–7874.
- [53] H. Hojjati, M. Sadeghi, and N. Armanfard, "Multivariate time-series anomaly detection with temporal self-supervision and graphs: Application to vehicle failure prediction," in *Proc. Eur. Conf. Mach. Learn. Princ. Pract. Knowl. Discov. Databases*, 2023, pp. 242–259.
- [54] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.
- [55] X. Zhu, C. Lei, H. Yu, Y. Li, J. Gan, and S. Zhang, "Robust graph dimensionality reduction," in *Proc. Int. Joint Conf. Artif. Intell.*, 2018, pp. 3257–3263.
- [56] H. Qu, L. Li, Z. Li, and J. Zheng, "Supervised discriminant isomap with maximum margin graph regularization for dimensionality reduction," *Expert Syst. Appl.*, vol. 180, 2021, Art. no. 115055.
- [57] H. Yang, K. Ma, and J. Cheng, "Rethinking graph regularization for graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4573–4581.
- [58] I. Ahmed, T. Galoppo, X. Hu, and Y. Ding, "Graph regularized autoencoder and its application in unsupervised anomaly detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 8, pp. 4110–4124, Aug. 2022.
- [59] J. Zeng, J. Pang, W. Sun, and G. Cheung, "Deep graph Laplacian regularization for robust denoising of real images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2019.
- [60] S. Rey and A. G. Marques, "Robust graph-filter identification with graph denoising regularization," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2021, pp. 5300–5304.
- [61] X. Li, H. Fan, and J. Liu, "Noise-aware clustering based on maximum entropy criterion and adaptive graph regularization," *Inf. Sci.*, vol. 626, pp. 42–59, 2023.
- [62] S. Tang et al., "Modeling multivariate biosignals with graph neural networks and structured state space models," in *Proc. Conf. Health, Inference, Learn.*, 2023, pp. 50–71.

- [63] Z. Kang, H. Pan, S. C. Hoi, and Z. Xu, "Robust graph learning from noisy data," *IEEE Trans. Cybern.*, vol. 50, no. 5, pp. 1833–1843, May 2020.
- [64] P. Berger, G. Hannak, and G. Matz, "Efficient graph learning from noisy and incomplete data," *IEEE Trans. Signal Inf. Process. Netw.*, vol. 6, pp. 105–119, 2020.
- [65] X. Du et al., "Noise-robust graph learning by estimating and leveraging pairwise interactions," *Trans. Mach. Learn. Res.*, 2023. [Online]. Available: <https://openreview.net/forum?id=r7imkFEAQb>
- [66] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–38, 2021.
- [67] Z. Zhang and M. Sabuncu, "Generalized cross entropy loss for training deep neural networks with noisy labels," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 8792–8802.
- [68] X. Teng, M. Yan, A. M. Ertugrul, and Y.-R. Lin, "Deep into hypersphere: Robust and unsupervised anomaly discovery in dynamic networks," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 2724–2730.
- [69] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato, "Grammar variational autoencoder," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1945–1954.
- [70] W. Zhang, C. Zhang, and F. Tsung, "Grelen: Multivariate time series anomaly detection from the perspective of graph relational learning," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 2390–2397.
- [71] A. Vaswani, "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [72] W. Chen, L. Tian, B. Chen, L. Dai, Z. Duan, and M. Zhou, "Deep variational graph convolutional recurrent network for multivariate time series anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 3621–3633.
- [73] I. Kobyzev, S. J. Prince, and M. A. Brubaker, "Normalizing flows: An introduction and review of current methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 11, pp. 3964–3979, Nov. 2021.
- [74] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2828–2837.
- [75] F. Xia, X. Chen, S. Yu, M. Hou, M. Liu, and L. You, "Coupled attention networks for multivariate time series anomaly detection," *IEEE Trans. Emerg. Topics Comput.*, vol. 12, no. 1, pp. 240–253, Jan./Mar. 2024.
- [76] T. K. K. Ho and N. Armanfard, "Contaminated multivariate time-series anomaly detection with spatio-temporal graph conditional diffusion models," 2024, *arXiv:2308.12563*.
- [77] X. Jiang et al., "Softpatch: Unsupervised anomaly detection with noisy data," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 15433–15445.
- [78] Y. Tashiro, J. Song, Y. Song, and S. Ermon, "CSDI: Conditional score-based diffusion models for probabilistic time series imputation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 24804–24816.
- [79] K. Rasul, C. Seward, I. Schuster, and R. Vollgraf, "Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8857–8868.
- [80] H. Liang, L. Song, J. Du, X. Li, and L. Guo, "Consistent anomaly detection and localization of multivariate time series via cross-correlation graph-based encoder-decoder gan," *IEEE Trans. Instrum. Meas.*, vol. 71, 2022, Art. no. 3504210.
- [81] L. Zhou, Q. Zeng, and B. Li, "Hybrid anomaly detection via multihead dynamic graph attention networks for multivariate time series," *IEEE Access*, vol. 10, pp. 40967–40978, 2022.
- [82] D. Guo, Z. Liu, and R. Li, "Regraphgan: A graph generative adversarial network model for dynamic network anomaly detection," *Neural Netw.*, vol. 166, pp. 273–285, 2023.
- [83] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 841–850.
- [84] S. Han and S. S. Woo, "Learning sparse latent graph representations for anomaly detection in multivariate time series," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 2977–2986.
- [85] D. Eswaran and C. Faloutsos, "Sedanspot: Detecting anomalies in edge streams," in *Proc. IEEE Int. Conf. Data Mining*, 2018, pp. 953–958.
- [86] J.-X. Zhong, N. Li, W. Kong, S. Liu, T. H. Li, and G. Li, "Graph convolutional label noise cleaner: Train a plug-and-play action classifier for anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. pattern Recognit.*, 2019, pp. 1237–1246.
- [87] S. Bhatia, B. Hooi, M. Yoon, K. Shin, and C. Faloutsos, "MIDAS: Microcluster-based detector of anomalies in edge streams," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 3242–3249.
- [88] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.
- [89] T. Zhao, B. Ni, W. Yu, Z. Guo, N. Shah, and M. Jiang, "Action sequence augmentation for early graph-based anomaly detection," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 2668–2678.
- [90] P. Boniol and T. Palpanas, "Series2graph: Graph-based subsequence anomaly detection for time series," 2022, *arXiv:2207.12208*.
- [91] C. Ding, S. Sun, and J. Zhao, "MST-GAT: A multimodal spatial-temporal graph attention network for time series anomaly detection," *Inf. Fusion*, vol. 89, pp. 527–536, 2023.
- [92] A. Karami, T. K. K. Ho, and N. Armanfard, "Graph-jigsaw conditioned diffusion model for skeleton-based video anomaly detection," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2025, pp. 4237–4247.
- [93] Y. Zhu et al., "A survey on graph structure learning: Progress and opportunities," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 1–10.
- [94] N. Li, J.-X. Zhong, X. Shu, and H. Guo, "Weakly-supervised anomaly detection in video surveillance via graph convolutional label noise cleaning," *Neurocomputing*, vol. 481, pp. 154–167, 2022.
- [95] Z. Chen, M. Ma, T. Li, H. Wang, and C. Li, "Long sequence time-series forecasting with deep learning: A survey," *Inf. Fusion*, vol. 97, 2023, Art. no. 101819.
- [96] Y. Yao et al., "DoTA: Unsupervised detection of traffic anomaly in driving videos," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 444–459, Jan. 2023.
- [97] K. Zhang et al., "Self-supervised learning for time series analysis: Taxonomy, progress, and prospects," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 10, pp. 6775–6794, Oct. 2024.
- [98] H. Hojjati, T. K. K. Ho, and N. Armanfard, "Self-supervised anomaly detection in computer vision and beyond: A survey and outlook," *Neural Netw.*, vol. 172, 2024, Art. no. 106106.
- [99] L. Zheng, Z. Li, J. Li, Z. Li, and J. Gao, "AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 4419–4425.
- [100] Y. Liu et al., "Anomaly detection in dynamic graphs via transformer," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12081–12094, Dec. 2023.
- [101] D. Purwanto, Y.-T. Chen, and W.-H. Fang, "Dance with self-attention: A new look of conditional random fields on anomaly detection in videos," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 173–183.
- [102] S. Chang, Y. Li, J. S. Shen, J. Feng, and Z. Zhou, "Contrastive attention for video anomaly detection," *IEEE Trans. Multimedia*, vol. 24, pp. 4067–4076, 2022.
- [103] K. Doshi and Y. Yilmaz, "Towards interpretable video anomaly detection," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2023, pp. 2655–2664.
- [104] H. Chen, X. Mei, Z. Ma, X. Wu, and Y. Wei, "Spatial-temporal graph attention network for video anomaly detection," *Image Vis. Comput.*, vol. 131, 2023, Art. no. 104629.
- [105] N. Laptev, S. Amizadeh, and Y. Billawala, "S5-A labeled anomaly detection dataset, version 1.0 (16M)," 2015. [Online]. Available: <https://webscope.sandbox.yahoo.com/catalog.php>
- [106] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [107] A. P. Mathur and N. O. Tippenhauer, "SWaT: A water treatment testbed for research and training on ICS security," in *Proc. Int. Workshop Cyber-Physical Syst. Smart Water Netw.*, 2016, pp. 31–36.
- [108] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "WADI: A water distribution testbed for research in the design of secure cyber physical systems," in *Proc. 3rd Int. Workshop Cyber-Phys. Syst. Smart Water Netw.*, 2017, pp. 25–28.
- [109] S. D. Bay, D. Kibler, M. J. Pazzani, and P. Smyth, "The UCI KDD archive of large data sets for data mining research and experimentation," *ACM SIGKDD Explorations Newslett.*, vol. 2, no. 2, pp. 81–85, 2000.
- [110] J. T. Brockmann, M. Rudolph, B. Rosenhahn, and B. Wandt, "The voraus-AD dataset for anomaly detection in robot applications," *IEEE Trans. Robot.*, vol. 40, pp. 438–451, 2024.

- [111] A. L. Goldberger et al., “Physiobank, physiotoolkit, and physionet: Components of a new research resource for complex physiologic signals,” *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.
- [112] A. Taddei et al., “The European ST-T database: Standard for evaluating systems for the analysis of ST-T changes in ambulatory electrocardiography,” *Eur. Heart J.*, vol. 13, pp. 1164–1172, 1992.
- [113] P. Wagner et al., “PTB-XL, a large publicly available electrocardiography dataset,” *Sci. Data*, no. 1, 2020, Art. no. 154.
- [114] F. Liu et al., “An open access database for evaluating the algorithms of electrocardiogram rhythm and morphology abnormality detection,” *J. Med. Imag. Health Inform.*, vol. 8, no. 7, pp. 1368–1373, 2018.
- [115] V. Shah et al., “The temple university hospital seizure detection corpus,” *Front. Neuroinform.*, vol. 12, 2018, Art. no. 83.
- [116] T. Opsahl and P. Panzarasa, “Clustering in weighted networks,” *Social Netw.*, vol. 31, no. 2, pp. 155–163, 2009.
- [117] R. Rossi and N. Ahmed, “The network data repository with interactive graph analytics and visualization,” in *Proc. AAAI Conf. Artif. Intell.*, 2015, pp. 4292–4293.
- [118] M. De Choudhury, H. Sundaram, A. John, and D. D. Seligmann, “Social synchrony: Predicting mimicry of user actions in online social media,” in *Proc. Int. Conf. Comput. Sci. Eng.*, 2009, pp. 151–158.
- [119] S. Kumar, F. Spezzano, V. Subrahmanian, and C. Faloutsos, “Edge weight prediction in weighted signed networks,” in *Proc. IEEE 16th Int. Conf. Data Mining*, 2016, pp. 221–230.
- [120] S. Kumar, B. Hooi, D. Makhija, M. Kumar, C. Faloutsos, and V. Subrahmanian, “REV2: Fraudulent user prediction in rating platforms,” in *Proc. 11th ACM Int. Conf. Web Search Data Mining*, 2018, pp. 333–341.
- [121] B. Zhang, R. Liu, D. Massey, and L. Zhang, “Collecting the Internet as-level topology,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 1, pp. 53–61, 2005.
- [122] W. Sultani, C. Chen, and M. Shah, “Real-world anomaly detection in surveillance videos,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6479–6488.
- [123] P. Wu et al., “Not only look, but also listen: Learning multimodal violence detection under weak supervision,” in *Proc. 16th Eur. Conf. Comput. Vis.*, 2020, pp. 322–339.
- [124] W. Luo, W. Liu, and S. Gao, “A revisit of sparse coding based anomaly detection in stacked RNN framework,” in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 341–349.
- [125] W. Li, V. Mahadevan, and N. Vasconcelos, “Anomaly detection and localization in crowded scenes,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 1, pp. 18–32, Jan. 2014.
- [126] D. Wagner, T. Michels, F. C. Schulz, A. Nair, M. Rudolph, and M. Kloft, “TimeSeAD: Benchmarking deep multivariate time-series anomaly detection,” *Trans. Mach. Learn. Res.*, 2023. [Online]. Available: <https://openreview.net/forum?id=iMmsCIOJS>
- [127] A. Koran, H. Hojjati, and N. Armanfard, “Unveiling the flaws: A critical analysis of initialization effect on time series anomaly detection,” 2024, *arXiv:2408.06620*.
- [128] T. Lai, T. K. K. Ho, and N. Armanfard, “Open-set multivariate time-series anomaly detection,” in *Proc. Eur. Conf. Artif. Intell.*, 2024, pp. 2003–2010.
- [129] E. C. Erkus and V. Purutcuoglu, “A new collective anomaly detection approach using pitch frequency and dissimilarity: Pitchy anomaly detection (PAD),” *J. Comput. Sci.*, vol. 72, 2023, Art. no. 102084.
- [130] H. Xu et al., “Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications,” in *Proc. World Wide Web Conf.*, 2018, pp. 187–196.
- [131] L. Shen, Z. Li, and J. Kwok, “Timeseries anomaly detection using temporal hierarchical one-class network,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 13016–13026.
- [132] C. U. Carmona, F.-X. Aubet, V. Flunkert, and J. Gasthaus, “Neural contextual anomaly detection for time series,” in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 2843–2851.
- [133] W. Zhang, C. Zhang, and F. Tsung, “GRELEN: Multivariate time series anomaly detection from the perspective of graph relational learning,” in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 2390–2397.
- [134] H. Zhang, Z. Wu, Z. Wang, Z. Chen, and Y.-G. Jiang, “Prototypical residual networks for anomaly detection and localization,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 16281–16291.
- [135] X. Yao, R. Li, J. Zhang, J. Sun, and C. Zhang, “Explicit boundary guided semi-push-pull contrastive learning for supervised anomaly detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 24490–24499.
- [136] J. Davis and M. Goadrich, “The relationship between precision-recall and ROC curves,” in *Proc. 23rd Int. Conf. Mach. Learn.*, 2006, pp. 233–240.
- [137] C. Ding, G. Pang, and C. Shen, “Catching both gray and black swans: Open-set supervised anomaly detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 7388–7398.



**Thi Kieu Khanh Ho** received the MSc degree from the Gwangju Institute of Science and Technology, in 2020. She is working toward the PhD degree with iSMART Lab, Department of Electrical and Computer Engineering, McGill University and Mila-Quebec AI Institute. From 2017 to 2018, she was a research assistant with Seoul National University Hospital. From 2020 to 2021, she was a researcher with the Korea National University of Transportation. Her research focuses on time-series anomaly detection, graph anomaly detection, and self-supervised learning.



**Ali Karami** received the BSc degree in electrical engineering from the University of Tehran, Iran, in 2021. He is working toward the master's degree with the Department of Electrical and Computer Engineering, McGill University. He is also affiliated with Mila-Quebec AI Institute. His research interest includes graph learning, anomaly detection, and self-supervised learning.



**Narges Armanfard** received the PhD degree in artificial intelligence from McMaster University and the postdoctoral degree in artificial intelligence from the University of Toronto. She is the founder and principal investigator with iSMART Lab. She holds the position of associate professor with the Department of Electrical and Computer Engineering, McGill University, and with Mila-Quebec AI Institute. She is also affiliated with McGill Centre for Intelligent Machines (CIM), McGill initiative in Computational Medicine (MiCM), and McGill Institute for Aerospace Engineering (MIAE). Her research focus lies in developing innovative algorithms for various domains such as time-series analysis, computer vision, reinforcement learning, and representation learning for tasks including data clustering, classification, and anomaly detection.